Session 2.1

The Learning Problem: a Handwritten Digit, and the Neuron

What it means to learn a function from examples, and the smallest unit that does it



Redistribution prohibited without permission
@Zhigang Feng

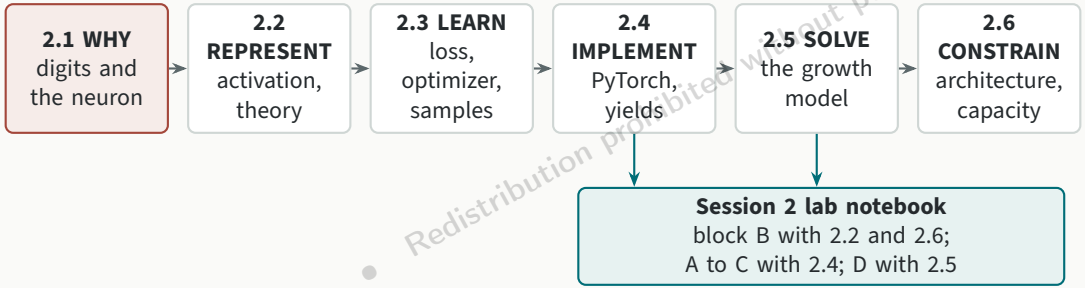
Zhigang Feng

What You Will Be Able to Do After Session 2

- **Read a network as a function**: write $f_\theta(x)$, count its parameters, and say what its architecture assumes about the input.
- **Check one training step by hand against autograd**: forward, loss, backward, update (Deck 2.4).
- **Write the PyTorch training loop** and check it on a problem whose answer you already know.
- **Run an honest forecast comparison**: splits by date, one information set, a random-walk baseline.
- **Solve a growth model with a network**, and show with held-out tests that the solution is right.
- **Throughout: a small training loss is a claim about the training sample, and nothing more.**

© Zhigang Feng · Redistribution prohibited without permission

Session 2 in One Map



© Thibaut Feng
Redistribution prohibited without permission

- The running example for 2.1 to 2.3 is one picture: a handwritten digit.
- The economic applications are in 2.4 (a forecast) and 2.5 (a model solved).
- **One habit in all six decks: test a method first where the answer is known.**

What Session 1 Already Gave Us, and What It Only Named

- **Four verbs** for what AI does in economics: measure, predict, identify, compute (Session 1.2). Session 2 is about *predict* and *compute*.
- **Three kinds of learning**: supervised (inputs with labels), unsupervised (inputs only), reinforcement (learning by acting). Session 2 is supervised; reinforcement is Session 3.
- **Empirical risk minimisation**: choose θ to minimise the average loss $\frac{1}{n}\sum_i \ell\big(f_\theta(x_i), y_i\big)$ over a sample (Session 1.3). OLS is the linear, squared-loss case.
- Session 1 only *named* the rule that makes this honest: judge a model on data it never saw. Deck 2.3 turns it into a procedure, and every result after it is reported that way.
- **Today starts one step earlier: with a problem no formula solves.**

© Zhitang Feng · Redistribution prohibited without permission

Session 2.1

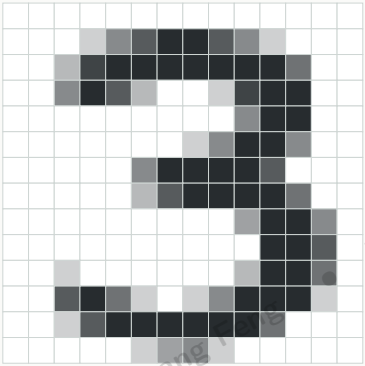
A Handwritten Digit

One picture carries the ideas of Decks 2.1 to 2.3



© Zhigang Feng Redistribution prohibited without permission

A Handwritten Digit

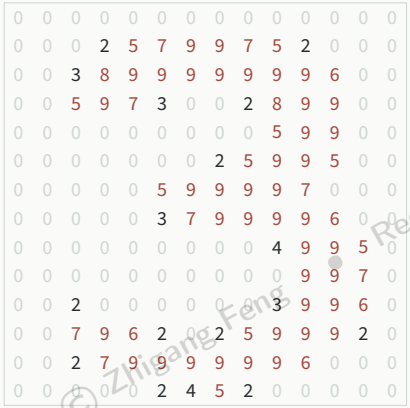


- You read this as a “3” at once, and you cannot say how.
- The benchmark is **MNIST**: 70,000 digits written by US Census Bureau staff and high-school students, each a 28×28 grey-scale image with its label.
- Drawn here at 14×14, so that every pixel can be seen.
- **Nobody writes that function down. It is learned from labelled examples.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, “Gradient-based learning applied to document recognition”, *Proc. IEEE* 86(11), 1998.

The Same Thing, Written Down



- To a computer the image is a table of ink levels: 0 is blank paper, 9 is full ink (0 to 255 in the original files).
- Read row after row, the table is a vector: 196 numbers here, $28 \times 28 = 784$ in MNIST. That vector is the entire input.
- Flattening forgets which pixels are neighbours. Deck 2.6 puts that back.
- **An image is a point in a space with one dimension per pixel: 196 here, 784 in MNIST.**

© Zhigang Feng Redistribution prohibited without permission

The Learning Problem: from 784 Numbers to Ten Classes



© Thiago Pérez — Redistribution prohibited without permission

- **Data:** 60,000 labelled pairs (x_i, y_i) for training, 10,000 held out for testing.
- **Model:** a family of functions f_θ indexed by a parameter vector θ.
- **Loss:** small when $\hat{p}$ puts high probability on the true digit.
- **Learning:** choose θ to make the average training loss small, then measure on the held-out images.
- **The econometrician's problem with a richer f_θ: a specification, a criterion, an estimator.**

Try Writing the Rule for a “3”



four people, one digit

- A first rule: “two open curves, stacked”. Now define *curve*, *open* and *stacked* in ink levels.
- Slant, stroke width, size and position change all 784 numbers and none changes the digit.
- **Polanyi’s paradox**: we know more than we can tell.
- **When the rule cannot be written, give examples and let the rule be estimated.**

© Zhigang Feng
Redistribution prohibited without permission

Source: M. Polanyi, *The Tacit Dimension*, 1966; D. Autor, “Polanyi’s paradox and the shape of employment growth”, NBER WP 20485, 2014.

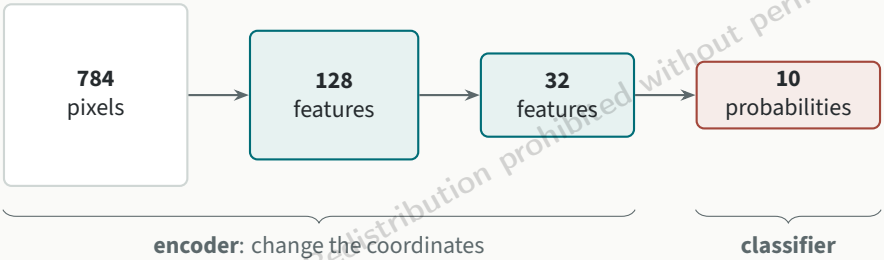
Why It Is Possible at All: Structure

- There are 256^{784} possible 28×28 images. Almost all of them look like television static.
- Images of digits are a vanishing fraction of that space, and they are not scattered at random: strokes are continuous, connected and few.
- The **manifold hypothesis**: real high-dimensional data lie near a surface of much lower dimension inside the space they are recorded in.
- Macroeconomists use the same idea. Krusell and Smith (1998) replace an infinite-dimensional wealth distribution by its mean, because in equilibrium the distribution moves in very few directions.
- **Learning in high dimension works when the data have low-dimensional structure, and fails when they do not.**

© Zhiyang Feng • Redistribution prohibited without permission

Source: P. Krusell and A. Smith, "Income and wealth heterogeneity in the macroeconomy", *JPE* 106(5), 1998.

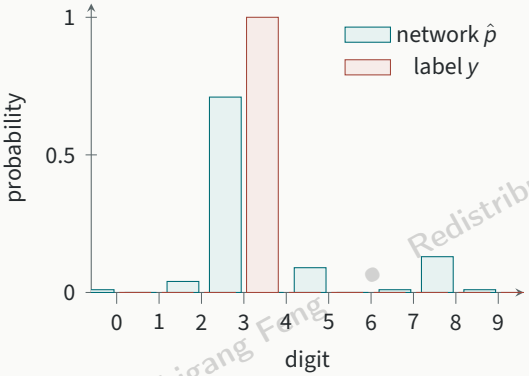
The Encoder View



© Zhirang Feng Redistribution prohibited without permission

- Each layer re-describes the input in fewer, more useful numbers. The last layer is an ordinary multinomial logit on those numbers.
- The encoder and the classifier are estimated *jointly*, by the same criterion.
- In econometric terms: the regressors are estimated, not chosen by the researcher.
- **A deep network is a logit whose explanatory variables are learned.**

Labels, One-Hot Targets, and a Probability for Each Digit



© Zhigang Feng. Redistribution prohibited without permission

- The label "3" is a **one-hot** vector: a 1 in position 3, zeros elsewhere.
- The **softmax** turns ten scores z_j into probabilities, $\hat{p}_j = e^{z_j} / \sum_k e^{z_k}$: McFadden's multinomial logit, with scores as utilities.
- The loss is the negative log-likelihood of the true digit: $-\log 0.71 = 0.34$ here.
- **Classification is maximum likelihood for a discrete-choice model.**

What the Input's Structure Decides

Input	Structure it carries	Architecture	Economic example
image	neighbours matter	convolutional	night lights
sequence	order matters	recurrent, transformer	policy statements
set, network	order is irrelevant	graph network	households
state vector	none assumed	dense	(k, z) in a growth model

- Decks 2.1 to 2.5 use the dense network, which assumes nothing. Deck 2.6 returns to the other rows.
- **An architecture is an assumption about the input, written into the function.**

© Zixiang Feng Redistribution prohibited without permission

Session 2.1

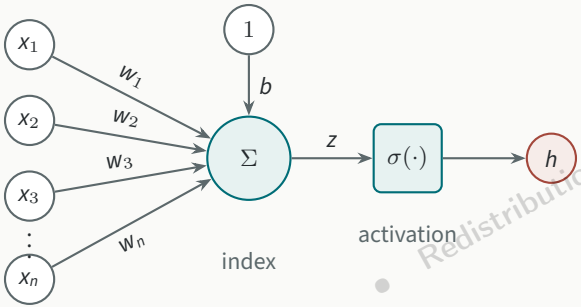
The Neuron

A regression with a nonlinear output



Redistribution prohibited without permission
© Zhigang Feng

A Neuron Is a Regression with a Nonlinear Output

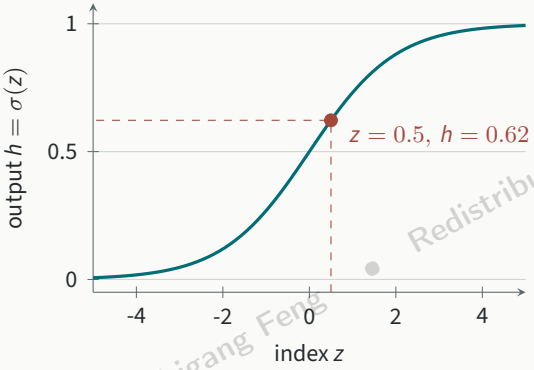


- Step 1, a linear index: $z = w^\top x + b$. The **weights** w are slopes; the **bias** b is the intercept.
- Step 2, a fixed nonlinear function, the **activation**: $h = \sigma(z)$.
- With $\sigma(z) = 1/(1 + e^{-z})$ this is exactly a logit model.
- The parameters are $\theta = (w, b)$: $n + 1$ numbers.
- **One neuron is a generalised linear model. Nothing more.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Miao & Feng, *AI for Economists*, §9.1.

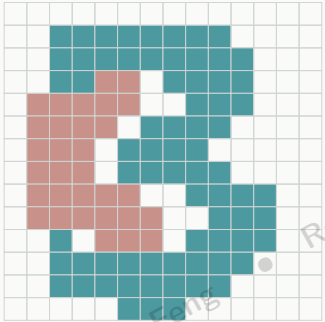
One Neuron, with Numbers



© Zhigang Feng
Redistribution prohibited without permission

- Inputs $x = (1.0,\ 0.5,\ 0.0)$; weights $w = (0.8,\ -0.4,\ 0.3)$; bias $b = -0.1$.
- Index: $z = 0.8(1.0) - 0.4(0.5) + 0.3(0.0) - 0.1 = 0.5$.
- Output: $h = \sigma(0.5) = 1/(1 + e^{-0.5}) = 0.62$.
- Read it as a probability: the neuron is "62% on".
- The third input is zero, so w_3 played no part. **A weight matters only where its input is active.**

What the Weights Are: a Template Laid over the Image

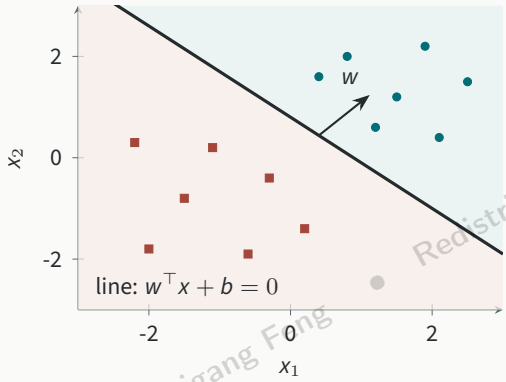


teal: positive weight **red**: negative weight

- A neuron reading the image has one weight per pixel, so its weights can be drawn as an image.
- $w^\top x$ is large when ink falls on positive weights and stays off negative ones.
- This one rewards ink where a “3” has strokes and penalises ink on the left, where an “8” closes its loops.
- **One neuron asks one question of the input: how much do you look like my template?**

© Zheng Feng Redistribution prohibited without permission

One Neuron Draws One Line



© Zhigang Feng
Redistribution prohibited without permission

- $\sigma(z) > 0.5$ exactly when $z = w^\top x + b > 0$.
- The neuron switches on in a half-space whose edge is the **hyperplane** $w^\top x + b = 0$: a line in two dimensions.
- w is perpendicular to the line; b slides it. The activation sets how sharply the output changes across the line, and does not bend it.
- **One neuron can separate only what a single straight cut can separate.**

An Economist Has Met This Before: Logit

- **Binary choice.** $\Pr(y = 1 \mid x) = \sigma(w^\top x + b)$ is the logit model; $w^\top x + b$ is the difference in systematic utility.
- **Estimation.** Maximum likelihood on the logit is minimising **cross-entropy**, the loss used to train classifiers. Same criterion, different name.
- **Vocabulary.** weights = coefficients; bias = intercept; activation = link function; training = estimation; inference = prediction.
- **What is new** is not the unit. It is stacking many units so that the regressors of the last one are themselves estimated.
- **Everything you know about specification, identification and out-of-sample testing still applies.**

© Zhiyang Feng · Redistribution prohibited without permission

Source: D. McFadden, "Conditional logit analysis of qualitative choice behavior", in *Frontiers in Econometrics*, 1974.

Takeaways, and What Deck 2.2 Opens

- **A learning problem** is data, a family f_θ, a loss, and a rule for judging the result.
- **An image is a vector**, and learning works because real data occupy a small, structured part of the space.
- **A neuron is a logit**: a linear index through a fixed nonlinearity, $h = \sigma(w^\top x + b)$.
- **Alone it draws one line**, and one line is not enough for the digits or for XOR.
- **The whole of a network's flexibility sits in that σ.** We have used it without looking at it.
- **Next, Deck 2.2: what σ is, why a layer without it buys nothing, and what a network with it can reach.**

© Zhirang Feng Redistribution prohibited without permission

Session 2.2

Nonlinearity: the Activation, XOR, and What a Network Can Approximate

Why a layer without σ buys nothing, and which functions the family with it can reach



Redistribution prohibited without permission
@Zhigang Feng

Zhigang Feng

What We Are Building, and What We Will Ask of It

- Deck 2.1 left one neuron, $h = \sigma(w^\top x + b)$, and one unexamined piece: the **activation** σ.
- **Question 1.** What is σ, and which ones will you actually meet?
- **Question 2.** Why is it needed at all — what fails in a layer without it, and what does a second layer fix that a wider first layer cannot?
- **Question 3.** How do kinks add up to a curve, and which functions can the family reach?
- Economists already choose flexible functional forms: translog, CES, polynomials, splines.
- **A network is one more flexible form. The difference is that it chooses its own basis.**

© Zhirang Feng · Redistribution prohibited without permission

Session 2.2

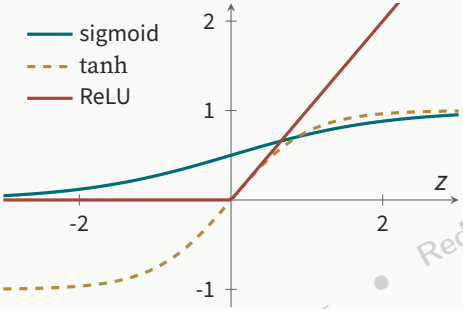
The Activation

The one nonlinear piece, and the three you will meet



© Zhigang Feng Redistribution prohibited without permission

The Activations You Will Actually Meet

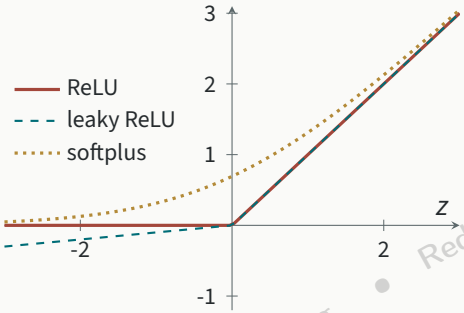


- **Sigmoid**, $1/(1+e^{-z})$, in $(0,1)$; **tanh**, in $(-1,1)$. Both **saturate**: flat in the tails.
- **ReLU**, $\max(0,z)$: zero on one side, the identity on the other. Slope exactly 0 or 1.
- ReLU for hidden layers; sigmoid where an output must be bounded.
- **No activation on a regression output: it would silently bound the prediction.**

Source: Miao & Feng, *AI for Economists*, §9.2, Table 9.2.

© Zhigang Feng
Redistribution prohibited without permission

ReLU Is a Hinge

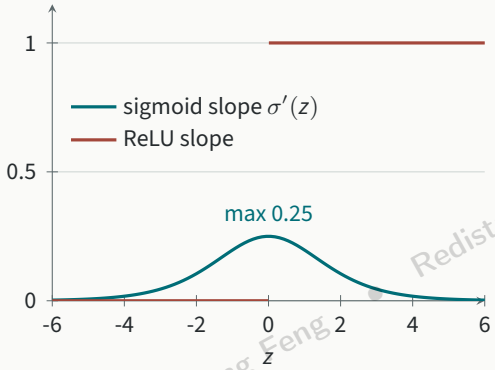


© Zhigang Feng
Redistribution prohibited without permission

- $\max(0, z)$ is a call option's payoff, and an occasionally binding constraint: inactive, then linear.
- **Leaky ReLU**: a small slope on the flat side, so a unit that switched off can return.
- **Softplus**, $\log(1 + e^z)$, rounds the corner, for models that need second derivatives.
- **Each hidden ReLU unit contributes one hinge. A network is a sum of hinges.**

Source: Miao & Feng, *AI for Economists*, §9.2, Fig. 9.4.

Saturation, and Where the Vanishing Gradient Begins



- The sigmoid's slope is $\sigma(z)(1-\sigma(z))$, never above 0.25.
- Training sends an error signal backwards through the layers (Deck 2.4, autograd), multiplied by this slope once per layer: at best 0.25^L.
- $L=3$: 0.016. $L=5$: 0.00098. $L=7$: 6×10^{-5}. The early layers barely move: the **vanishing gradient**.
- **ReLU's slope is 1 on its active side, so the signal survives depth. That is why it is the default.**

Source: Miao & Feng, *AI for Economists*, §9.2.

© Zhigang Feng Redistribution prohibited without permission

Session 2.2

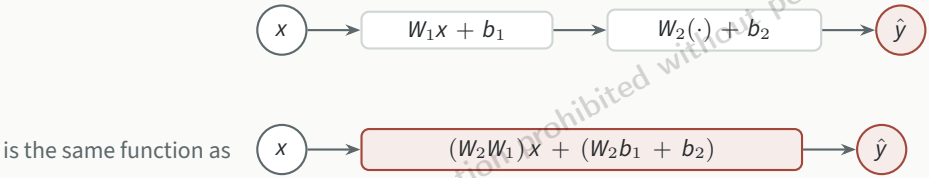
Why One Layer Is Not Enough

What collapses without σ, and what a hidden layer represents exactly



© Zhigang Feng
Redistribution prohibited without permission

Linear Stacks Stay Linear

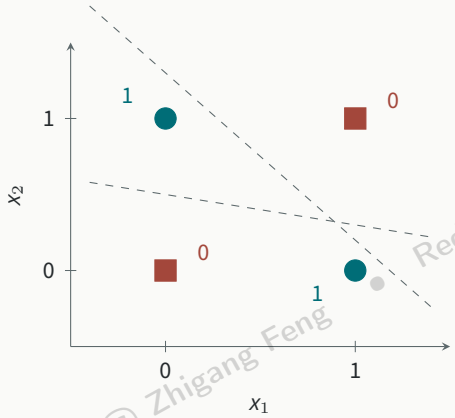


© Zhigang Feng — Redistribution prohibited without permission

- Remove the activation and compose two layers: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$.
- That is one linear map. A hundred linear layers are still one regression line.
- **Every bit of a network's flexibility comes from the activation between the linear maps.**

Source: Miao & Feng, *AI for Economists*, §9.1.

What One Neuron Cannot Do

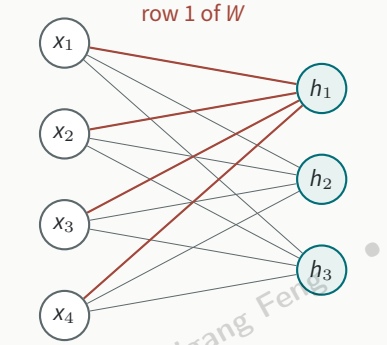


Source: M. Minsky and S. Papert, *Perceptrons*, MIT Press, 1969.

- **XOR**, exclusive or: the output is 1 when exactly one of two binary inputs is 1.
- The two 1s sit on one diagonal, the two 0s on the other. No straight line separates them: try the dashed ones.
- Minsky and Papert (1969) made this the standard objection to single-layer networks.
- **The cure is a second layer. That is the rest of this section.**

© Zhigang Feng
Redistribution prohibited without permission

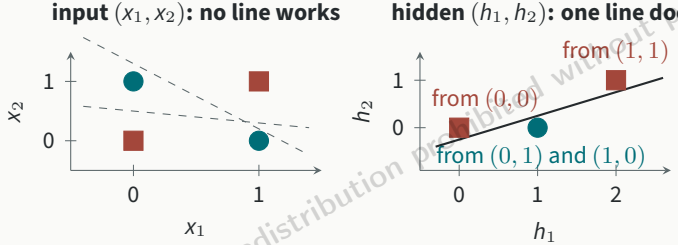
A Layer Is Many Neurons Reading the Same Input



- Put m neurons side by side. Each has its own weights and bias; all read the same x.
- Stack the weight vectors as rows of a matrix W ($m \times n$): $h = \sigma(Wx + b)$, with σ applied entry by entry.
- Here W is 3×4: 12 weights and 3 biases, 15 parameters.
- **A layer is a system of m regressions sharing their regressors.**

© Zhigang Feng Redistribution prohibited without permission

The Same Four Points, Moved by the Hidden Layer

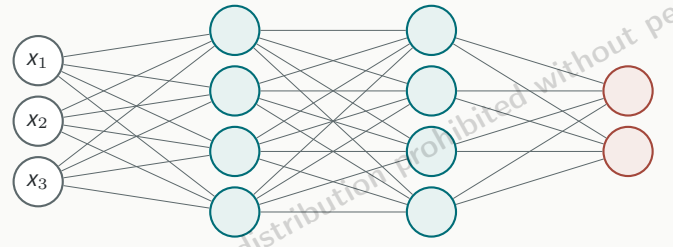


- **The same four points, drawn twice.** Same marks, same colours; only the axes change.
- Write $s = x_1 + x_2$. Then $h_1 = \max(0, s)$ and $h_2 = \max(0, s - 1)$: $(0,0) \mapsto (0,0)$, both $(0,1)$ and $(1,0) \mapsto (1,0)$, $(1,1) \mapsto (2,1)$.
- The drawn line is the output $\hat{y} = h_1 - 2h_2$: $0,\ 1,\ 1,\ 0$, *exact*, not approximate. **The hidden layer did not fit a curve — it moved the points until one line was enough.**

Source: Miao & Feng, *AI for Economists*, §9.2; Goodfellow, Bengio, Courville, *Deep Learning*, 2016, §6.1.

© Zhiqiang Feng · Redistribution prohibited without permission

The Object in One Line

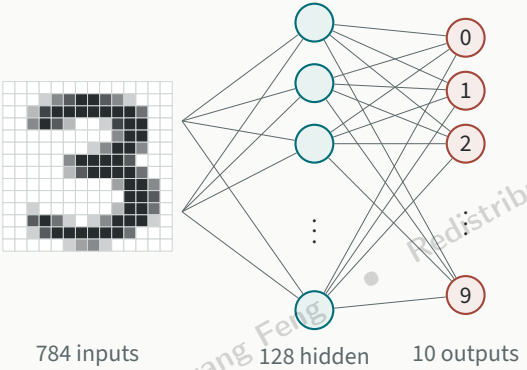


© Shigeki Fujii. Redistribution prohibited without permission

- A **feedforward network** (multilayer perceptron, MLP) composes layers: $f_\theta(x) = W_3\,\sigma\big(W_2\,\sigma(W_1 x + b_1) + b_2\big) + b_3$.
- θ collects every W_ℓ, b_ℓ. **Width**: units per layer. **Depth**: number of layers.
- **A network is a parametric family: alternate a linear map with a fixed nonlinearity.**

Source: Miao & Feng, *AI for Economists*, §9.1, eq. (9.6).

A Network for Digits: 784 → Hidden → 10



- Layer 1: 784×128 weights $+ 128$ biases $= 100{,}480$ parameters.
- Layer 2: $128 \times 10 + 10 = 1{,}290$. Total: **101,770**.
- Trained as in Deck 2.3, it classifies 97 to 98% of the 10,000 test images correctly.
- **More parameters than observations, and it still generalises. Deck 2.6 asks why.**

© Zhigang Feng. Redistribution prohibited without permission

Source: Accuracy: the 784–128–10 example of the TensorFlow beginner tutorial; the same one-hidden-layer design in LeCun et al. (1998), 300 units: 95.3%.

Session 2.2

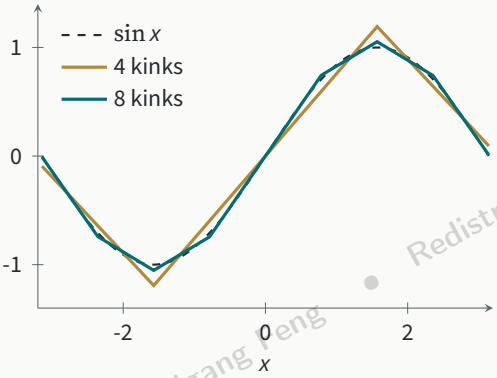
How a Network Bends a Straight Line

Approximating $\sin x$ out of hinges, and letting the hinges move



© Zhigang Feng Redistribution prohibited without permission

The Construction: an Approximation Built out of Kinks

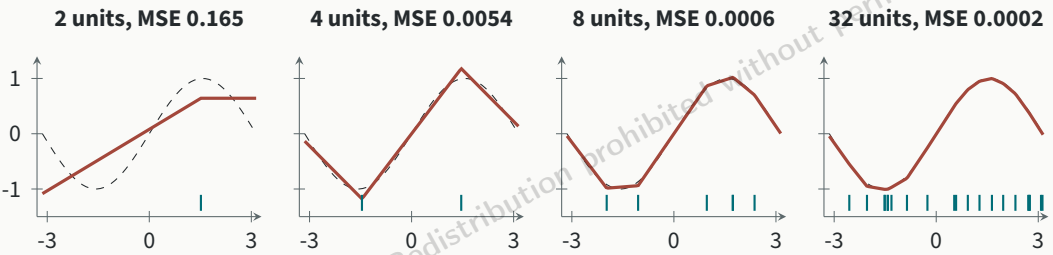


- One hidden ReLU layer, one input: $\hat{y}(x) = a_0 + \sum_{i=1}^{N} a_i \max(0,\, x - b_i)$.
- The shifts b_i are the **kinks**, where the slope may change; the a_i set the slope on each interval.
- Fix the b_i, evenly spaced, and $\hat{y}$ is linear in the a_i: ordinary least squares on a piecewise-linear basis.
- **Mean squared error 0.0059 with 4 kinks, 0.0003 with 8. There is no network here yet.**

Source: Session 2 lab notebook, block B; least squares on 121 points of $\sin x$.

© Zhigang Feng
Redistribution prohibited without permission

The Turn: Let the Kinks Choose Where to Go

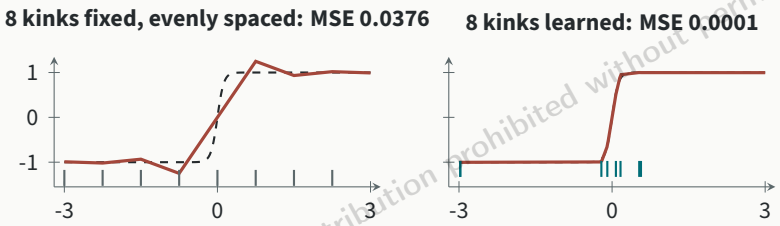


Redistribution prohibited without permission

- Train the b_i together with the a_i: a one-hidden-layer network. The teal ticks mark the learned kinks, and they are not evenly spaced.
- At 8 units fixed kinks by least squares (0.0003) beat training (0.0006); at 4 units training (0.0054) beat fixed (0.0059). A search, not a solve.
- **The basis is no longer chosen in advance. The approximant decides where to bend.**

Source: Session 2 lab notebook, block B; Adam, 3,000 steps, one seed.

Where Adaptivity Pays

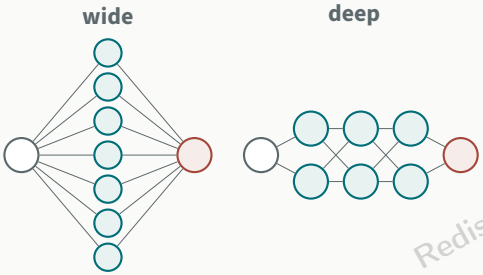


© Zhigang Feng Redistribution prohibited without permission

- $\tanh(8x)$: all the curvature near zero. Eight units either way.
- Six of the eight learned kinks crowd into an eighth of the domain; the fit is **396 times** better. On $\sin x$, whose curvature is spread evenly, training lost.
- A fixed basis is linear in its parameters: a vector space, a closed-form solve, a rate quotable in advance. Width-*m* networks are none of these.
- **A solve becomes a search. What that buys is priced next.**

Source: Adam, 6,000 steps, one seed, against least squares on the same eight kinks. Session 2 lab notebook, block B.

Width and Depth: Two Axes



- Width adds basis functions; depth composes them.
- Cost: 20 inputs, three hidden layers of 256 units:
  $5{,}376 + 65{,}792 + 65{,}792 + 257 = 137{,}217$
  parameters.
- Some functions need exponentially many units in two layers, polynomially many in three.
- **Depth pays when the target is a composition: data $\to$ beliefs $\to$ choice; households $\to$ aggregates.**

Source: R. Eldan and O. Shamir, "The power of depth for feedforward neural networks", COLT 2016; Miao & Feng, §9.1–9.2.

© Zhigang Feng
Redistribution prohibited without permission

Session 2.2

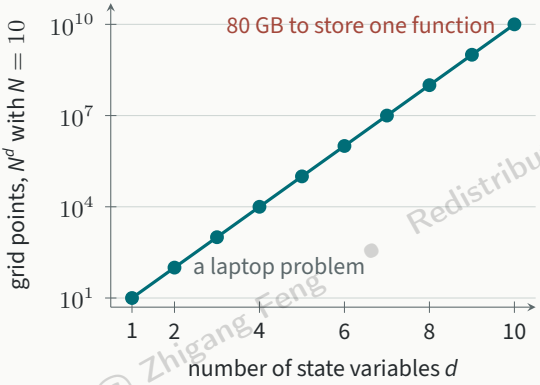
What the Theory Says

Existence, rates, and what neither one promises



© Zhigang Feng Redistribution prohibited without permission

The Curse of Dimensionality, in Arithmetic



- A grid with N points per state variable needs N^d points in d dimensions.
- With $N = 10$: 100 points at $d = 2$, a million at $d = 6$, ten billion at $d = 10$, which is 80 GB for one policy function.
- Wealth, income, age, housing and two aggregate shocks already make $d = 6$.
- **A method whose cost grows with the number of examples, not with N^d, changes what can be solved.**

© Zhigang Feng
Redistribution prohibited without permission

Universal Approximation, on a Compact Domain

- A **compact domain** is closed and bounded: $[0,1]^d$, or a capital range $[k_{\min}, k_{\max}]$.
- **Universal approximation**: for any continuous f on a compact domain and any $\varepsilon > 0$ there is a one-hidden-layer network $\hat{f}(x) = \sum_{j=1}^{m} v_j \, \sigma(w_j^{\top} x + b_j) + c$ with $\sup_x |f(x) - \hat{f}(x)| < \varepsilon$.
- Cybenko (1989) for sigmoids; Hornik (1991) for any bounded non-constant activation; Leshno et al. (1993) for any that is not a polynomial.
- Polynomials have the same property (Stone–Weierstrass), so universality does not distinguish a network.
- **The theorem says a good network exists on the domain. It says nothing outside the domain.**

© Zhiyang Feng · Redistribution prohibited without permission

Source: Cybenko, *Math. Control Signals Syst.* 2, 1989; Hornik, *Neural Networks* 4, 1991; Leshno, Lin, Pinkus, Schocken, *Neural Networks* 6, 1993.

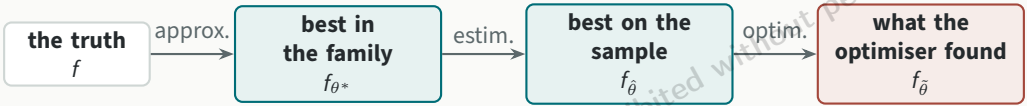
Four Things the Theorem Does Not Say

- **Not how many units.** "There exists m", and m may be astronomically large.
- **Not how to find the weights.** Good weights exist; finding them is a non-convex search that promises only a stationary point.
- **Not how much data.** Approximating a known function is different from estimating an unknown one from a finite sample.
- **Not that it will generalise.** Fitting the sample and fitting the function are different achievements.
- **Finding the weights** is the binding one. With 8 kinks, least squares reached mean squared error 0.0003; training the same 8 units reached 0.0006. A better member of the family existed and the optimiser did not find it.
- **Existence is a statement about the family, not about the network you trained.**

Source: Miao & Feng, *AI for Economists*, §9.1.

© Zhaonan Feng · Redistribution prohibited without permission

Three Errors: Approximation, Estimation, Optimization



- **Approximation error**: the family cannot reach f. Falls with width and depth. The theorems live here.
- **Estimation error**: a finite sample picks the wrong member. Falls with data, rises with flexibility.
- **Optimization error**: the algorithm stops short of the sample optimum. Deck 2.3.
- **Only the sum is observed, on held-out data. A bigger network lowers the first error and can raise the other two.**

Source: L. Bottou and O. Bousquet, “The tradeoffs of large scale learning”, NeurIPS 2007.

© Zhiheng Huang Redistribution prohibited without permission

Barron's Theorem, and the Class It Holds For

- Barron (1993): with m sigmoidal hidden units, on a ball of radius r,

$$\int_{B_r} \left(f - \hat{f}_m\right)^2 d\mu \;\le\; \frac{(2rC_f)^2}{m}, \qquad C_f = \int \|\omega\|\,|\tilde{f}(\omega)|\,d\omega,$$

  where $\tilde{f}$ is the Fourier transform of f.
- The **Barron class** is the set of functions with finite C_f: little weight at high frequencies.
- Double the units and the squared error halves, and the input dimension d is absent from the exponent.
- But C_f is an integral over $\mathbb{R}^d$. For many functions it grows with d; for others it is infinite.
- **The dimension did not vanish. It moved into the constant and into the narrowness of the class.**

Source: A. Barron, "Universal approximation bounds for superpositions of a sigmoidal function", *IEEE Trans. Inf. Theory* 39, 1993.

© Zhengling Qi · Redistribution prohibited without permission

The Rates, Side by Side

Functions	Approximator, size m	Best possible	Source
k times smooth	any continuous method	$\gtrsim m^{-k/d}$ (error)	DeVore et al. (1989)
Barron class	any basis fixed in advance	$\gtrsim m^{-2/d}$ (squared)	Barron (1993), Thm. 6
Barron class	one hidden layer	$\leq C/m$ (squared)	Barron (1993), Prop. 1
compositional	matching deep network	exponent free of d	Poggio et al. (2017)

© Zhiguo Feng · Redistribution prohibited without permission

- Rows 1–2 are the curse of dimensionality. Rows 3–4 escape it by assuming something about f.
- **The gap is between a basis fixed before seeing f and a basis adapted to f.**

Source: DeVore, Howard, Micchelli, *Manuscripta Math.* 63, 1989; Poggio et al., *Int. J. Autom. Comput.* 14, 2017.

Takeaways, and What Deck 2.3 Has to Supply

- **Without an activation, depth buys nothing.** With ReLU, each hidden unit is one hinge and a network is a sum of hinges.
- **A hidden layer changes the coordinates** until a line is enough; XOR shows this exactly, not approximately.
- **Fix the kinks and it is least squares; free them and the approximant chooses where to bend.**
- **Universal approximation is existence on a compact domain**: not size, not a recipe, not data, not generalisation.
- **Barron's rate has no d in the exponent**; the dimension lives in the constant and the class.
- **Every promise here was "a good network exists". Deck 2.3 is how to find one, and how to know when to stop.**

© Zhigang Feng · Redistribution prohibited without permission

Session 2.3

The Loss, the Optimizer, and Three Samples

What the loss assumes, how the optimizer uses a gradient, and when to stop



© Zhigang Feng Redistribution prohibited without permission

Zhigang Feng

Session 2.3

The Loss, and the Optimizer

What the criterion assumes, and what to do with a gradient that is noisy



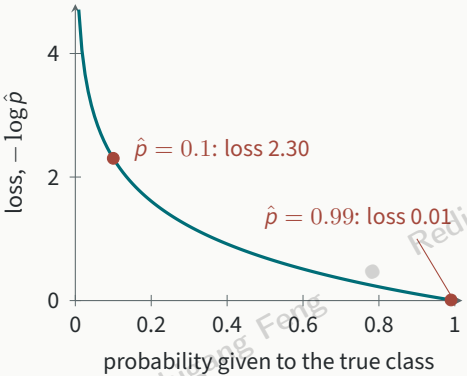
© Zhigang Feng Redistribution prohibited without permission

Loss I: Squared Error Is OLS

- For a continuous target, the mean squared error: $\mathcal{L} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)^2$.
- Minimising it is maximum likelihood under Gaussian errors. For a linear f_θ it *is* ordinary least squares.
- A loss is therefore an assumption about the noise, not a convention. Change the noise model and the loss changes: Poisson likelihood for counts, the check function for quantiles.
- In Deck 2.4 the target is next month's yield. In Deck 2.5 the "prediction" is an Euler-equation residual and the target is zero, so no data enter the loss at all.
- **Squared error asks for the conditional mean. If that is not what you want, it is the wrong loss.**

© Zhiguo Feng. Redistribution prohibited without permission

Loss II: Cross-Entropy

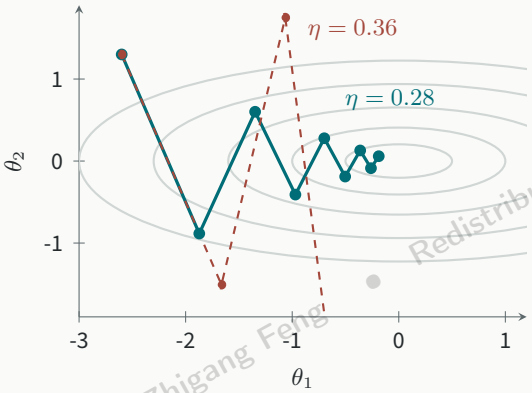


- A classifier returns a score z_k for each of K classes. The **softmax** turns scores into probabilities: $\hat{p}_k = e^{z_k} / \sum_j e^{z_j}$.
- **Cross-entropy** is the negative log-probability given to the true class, $\mathcal{L} = -\log \hat{p}_{k^\star}$.

© Zheng Feng
Redistribution prohibited without permission

Source: D. McFadden, "Conditional logit analysis of qualitative choice behavior", in *Frontiers in Econometrics*, 1974.

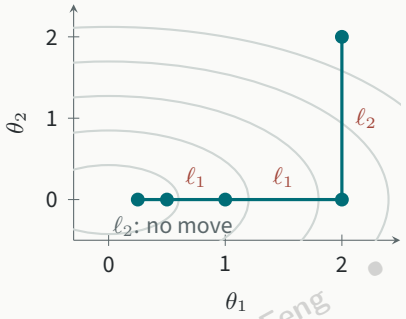
Gradient Descent: the Engine



© Zhigang Feng Redistribution prohibited without permission

- **Gradient descent**: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$. Step against the gradient.
- The picture is $\mathcal{L} = \frac{1}{2}(\theta_1^2 + 6\theta_2^2)$. The teal path zigzags across the steep direction and converges.
- The red path uses a step only slightly longer. It overshoots further each time and diverges.
- **The learning rate is limited by the steepest direction, and progress by the flattest.**

Why a Sampled Gradient Still Gets There



- **Stochastic gradient descent** (SGD): differentiate a random subset of the loss's terms at each step.
- Here $\ell_1 = \frac{1}{2}\theta_1^2$, $\ell_2 = \theta_2^2$, step 0.5: only the drawn coordinate moves.
- Unbiased: its expectation is proportional to the full gradient.
- **The path is not monotone and is not meant to be — and the goal is not the lowest training loss but a minimum that generalises.**

Source: L. Bottou, F. Curtis, J. Nocedal, *SIAM Rev.* 60(2), 2018; A. Choromanska et al., AISTATS 2015; T. Garipov et al., NeurIPS 2018.

© Zhigang Feng Redistribution prohibited without permission

Batch, Stochastic, Mini-Batch

Method	Terms per step	Cost of a step	Path
full batch	all n	high	smooth
stochastic	1	lowest	very noisy
mini-batch	32 to 1,024	one matrix product	noisy, averaged

- An **epoch** is one pass through the training data; an **iteration** is one update. With 40,000 observations and batches of 256, an epoch is 156 full batches.
- Why mini-batch: many updates per pass, activations that fit in memory, hardware built for matrix products.
- The gradient noise also helps the iterate leave saddle points and shallow minima.

© Zhirang Feng · Redistribution prohibited without permission

Session 2.3

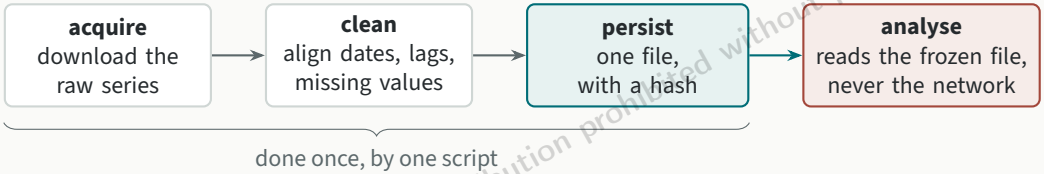
The Dataset, and Three Samples

Where the data come from, and the rule that makes a reported number mean something



© Zhigang Feng Redistribution prohibited without permission

The Data Workflow: Acquire, Clean, Persist — Once



© Zhiyang Feng · Redistribution prohibited without permission

- Statistical agencies revise. A notebook that downloads on every run gives different answers on different days.
- Separate the download from the analysis; the analysis then runs offline and gives the same numbers every time.
- **A result is reproducible only if its data are frozen.**

Train, Validation, Test: Three Jobs, Three Samples

training sample: choose the parameters θ	**validation**	**test**
the optimiser sees only this	choose the design: width, penalty, when to stop	report once, at the end

© Zheng Feng redistribution prohibited without permission

- A flexible model can fit its training sample as closely as you like. That says nothing about new data.
- **Validation** data choose between designs, so they are no longer a clean test.
- **Test** data are touched once. Their error estimates the error on future data.
- With time series the three samples are consecutive periods, never a shuffle.
- **Each sample does one job. A sample used for two jobs does neither honestly.**

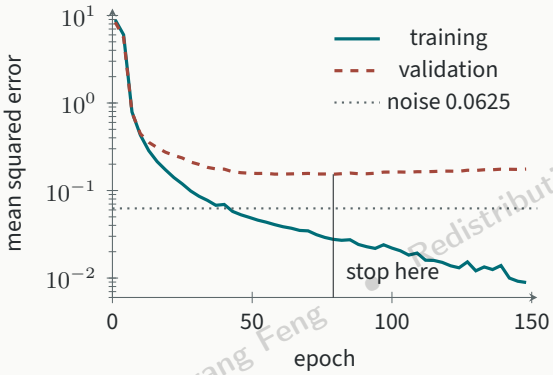
Leakage: Four Ways the Test Set Gets into Training

- **Leakage**: information from the test sample reaches the fitted model, so the test error is too good.
- **Shuffling a time series.** Neighbouring months are nearly identical; a shuffled split puts the answer for March next to the question for April.
- **Scaling with full-sample statistics.** The mean and standard deviation used to standardise the inputs must come from the training window alone.
- **Tuning on the test set.** Trying ten designs and reporting the best test error makes the test set a validation set.
- **The same template on both sides.** In the lab, sentences built as subject × phrase score 0.97 on a random split and 0.50 when whole phrases are held out.
- **Ask of every reported accuracy: what did the model already know about these cases?**

© Thiago Ferreira. All rights reserved. Redistribution prohibited without permission

Source: Accuracies: Session 2 lab notebook, appendix (logistic regression on word counts, 96 sentences).

Overfitting, Underfitting, and When to Stop

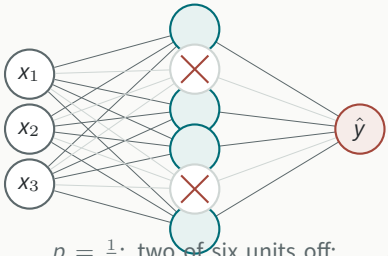


© Zhigang Feng Redistribution prohibited without permission

- **Underfitting**: both curves high. **Overfitting**: training loss falls while validation loss turns up.
- $n = 2{,}000$. Training loss ends at 0.0089, below the noise variance 0.0625: memorised noise.
- **Early stopping**: keep the best-validation parameters, near epoch 80.
- **A training loss below the noise floor is a diagnosis, not an achievement.**

Source: Recorded in the instructor's quantitative-macro course computations (2026); not recomputed in this session's lab.

Dropout, Step by Step



$p = \frac{1}{3}$: two of six units off; a new draw at every step

- **Dropout**: each step, each hidden unit is kept with probability $1 - p$, else zeroed: $\tilde{h} = m \odot h/(1-p)$, $m_j \sim \text{Bernoulli}(1-p)$, redrawn each step.
- The $1/(1-p)$ keeps $E[\tilde{h}] = h$; at prediction nothing is dropped and the full network runs once (`eval()` in Deck 2.4).
- $p = 0.1$ to 0.2 on hidden layers, never on the output.
- **A unit cannot count on any partner being present, so each must carry information on its own.**

© Zhigang Feng
Redistribution prohibited without permission

Source: N. Srivastava et al., "Dropout: a simple way to prevent neural networks from overfitting", *JMLR* 15, 2014.

Why Dropout Regularises

- With m hidden units there are 2^m thinned sub-networks; each step trains one of them, and all share one set of weights.
- At test time the full network approximates the average of those sub-networks: an ensemble for the price of one model.
- For a linear model, dropout on the inputs is a ridge penalty scaled by each regressor's second moment: an adaptive weight decay (Wager, Wang and Liang, 2013).
- It is noise on purpose: the network is trained to survive losing any one feature, as new data will make it.
- Cost: a noisier gradient and more steps; at $n = 50{,}000$ the next frame shows it is worth 1%.
- **Dropout pays where the data are too few to pin the weights down, and that is the economist's usual case.**

© Zhigang Feng · Redistribution prohibited without permission

Source: S. Wager, S. Wang, P. Liang, "Dropout training as adaptive regularization", *NeurIPS* 26, 2013; Srivastava et al. (2014).

Three Instruments Against Overfitting, and What They Are Worth

Strategy	best validation $n = 2{,}000$	best validation $n = 50{,}000$
none	0.1535	0.0788
weight decay 10^{-4}	0.1536	0.0804
dropout 0.1	0.1257	0.0780
dropout 0.2	0.1341	0.0792
both (0.1)	**0.1238**	0.0780

Noise floor 0.0625; 30 inputs, 17 pure noise.

- **Weight decay**: a penalty $\lambda\|\theta\|^2$, which is ridge regression.
- **Dropout**, from the previous two frames: a random share p of hidden units off at each step.
- Early stopping acts like a ridge penalty of about $1/(\eta T)$ after T steps.
- **Dropout pays where data are scarce: 18% at $n = 2{,}000$, 1% at $n = 50{,}000$. More data does more: 0.1535 to 0.0788.**

Source: Recorded in the instructor's quantitative-macro course computations (2026), median of three seeds; not recomputed in this session's lab.

© Zhigang Feng · Redistribution prohibited without permission

Takeaways, and What Is Still Missing

- **A loss is an assumption about the noise**: squared error is OLS under Gaussian errors; cross-entropy is the log-probability of the true class.
- **The optimizer steps against the gradient**: the learning rate is limited by the steepest direction, and a sampled gradient is unbiased, so mini-batches still get there.
- **Each sample does one job**: the optimiser sees the training data, the design is chosen on validation data, and the test data are touched once.
- **Generalisation is measured, not assumed**: a training loss below the noise floor is the mark of overfitting, and regularisation pays mainly when data are scarce. Deck 2.6 returns with a case the noise floor cannot diagnose at all.
- **Next, Deck 2.4: the same loop in PyTorch, first where the answer is known, then on next month's 10-year yield.**

© Lutang Peng · Redistribution prohibited without permission

Session 2.4

PyTorch, Curve Fitting, and Yield Forecasts

The training loop in code, a curve fitted, and a real yield series



© Zhigang Feng · Redistribution prohibited without permission

Zhigang Feng

What PyTorch Actually Is

tensors
arrays of numbers, as in NumPy

autograd
a record of every operation, so exact derivatives come back

nn.Module
a class that holds a network's parameters

optim
gradient-descent variants: SGD, Adam

- A scientific-computing library that became popular for deep learning, not the reverse.
- The first two pieces are the substance: write any scalar as a chain of array operations and its gradient is free.
- The last two are bookkeeping for parameters and update rules.
- **Everything in Deck 2.3 is four library calls. Today we make them.**

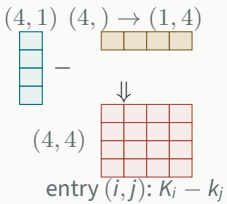
© Zhigang Feng Redistribution prohibited without permission

Source: A. Paszke et al., "PyTorch: an imperative style, high-performance deep learning library", *NeurIPS* 32, 2019.

Tensors: Shapes, Types, and the Broadcasting Bug

```
k = torch.linspace(0.5, 2.0, 4)
K = k.reshape(-1, 1)
print(k.shape, K.shape, K.dtype)
print((K - k).shape)
```

```
# shape (4,)
# shape (4, 1): four observations, one feature
# torch.Size([4]) torch.Size([4, 1]) torch.float32
# torch.Size([4, 4])   <- not what you meant
```

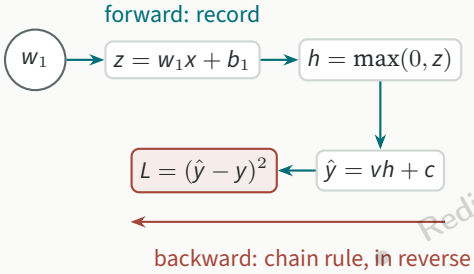


- A **tensor** is an array with a **shape** (4 rows, 1 column) and a **dtype** (32-bit floats).
- **Broadcasting** lines shapes up from the right, so a $(4,)$ is read as one row, $(1,4)$.
- Each size-1 axis is copied to match: the column across, the row down. Result: every K_i minus every k_j, silently.
- **Keep n observations of d variables as (n,d), and print the shape before you trust the number.**

Source: Session 2 lab notebook, block A1.

© Zhigang Feng
Reuse not prohibited without permission

Autograd: the Tape



- Mark a tensor `requires_grad=True` and PyTorch records every operation it enters: the **computation tape**.
- `backward()` walks the tape in reverse, one chain-rule step per operation: an exact derivative, not a finite difference.
- This is **autograd**: the chain rule, applied by the library.
- **You write the forward pass only.**

© Zhigang Feng
Redistribution prohibited without permission

`backward()`: One Step You Can Check by Hand

```
x, y = 2.0, 3.0
w1, b1, v, c = (torch.tensor(a, requires_grad=True) for a in (0.5, -0.1, 1.2, 0.3))
z = w1 * x + b1; h = torch.relu(z); yhat = v * h + c; L = (yhat - y) ** 2
L.backward()
# forward : z = 0.9000, h = 0.9000, yhat = 1.3800, L = 2.6244
# backward: dL/dv = -2.9160, dL/dc = -3.2400, dL/dw1 = -7.7760, dL/db1 = -3.8880
# update  : w1 -> 0.5778, v -> 1.2292;  loss 2.6244 -> 1.7488     (eta = 0.01)
```

- A four-parameter ReLU network on one observation. Each derivative is one chain-rule product, and the lab asserts every number against autograd.
- A second hidden unit with $z = -0.8 < 0$ receives a gradient of exactly 0.0: no signal, no learning.
- **If your hand derivative and autograd disagree, the hand derivative is wrong.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block A2; example from Miao & Feng, *AI for Economists*, §9.3.

The Training Loop Is Always the Same Five Lines

```
model = nn.Linear(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for step in range(300):
    prediction = model(X)                    # 1 forward
    loss = ((prediction - y) ** 2).mean()    # 2 loss
    optimizer.zero_grad()                    # 3 clear the old gradients
    loss.backward()                          # 4 backward
    optimizer.step()                         # 5 update
```

- Forward, loss, clear, backward, update. A language model with billions of parameters is trained by these five lines.
- Only line 1 (the model) and line 2 (the loss) change between applications.
- Fit a curve: *L* is the squared prediction error. Solve a model: *L* is the squared Euler residual, which is Deck 2.5.
- **Once derivatives are free, one algorithm covers both. Learn the loop once.**

Source: Session 2 lab notebook, block A3.

© Zhenlei Peng. Redistribution prohibited without permission

A Network Is a Class

```
class SimpleNet(nn.Module):
    def __init__(self, hidden=32, act=nn.ReLU):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(1, hidden), act(), nn.Linear(hidden, 1))
    def forward(self, x):
        return self.net(x)
```

- `nn.Module` is PyTorch's base class for anything with parameters. `__init__` declares the layers; `forward` says how an input flows through them.
- `nn.Linear(1, hidden)` is $Wx + b$ with W of shape hidden $\times$ 1; `nn.Sequential` composes layers in order.
- This is $f_\theta(x) = W_2\,\sigma(W_1 x + b_1) + b_2$ from Deck 2.2, with 32 hidden units.
- **The class is the specification. Read it as you would read a regression equation.**

Source: Session 2 lab notebook, block A4.

© Zhigang Feng · Redistribution prohibited without permission

Optimizers, `zero_grad`, and Seeds

```
torch.manual_seed(1); net = SimpleNet()                # same seed, same initial draw
optimizer = torch.optim.Adam(net.parameters(), lr=3e-3, weight_decay=weight_decay)
optimizer.zero_grad()                                  # 3 clear the old gradients
loss = loss_fn(net(X_tr), y_tr); loss.backward(); optimizer.step()
net.eval()                                             # evaluation mode
with torch.no_grad():                                  # no tape: nothing to differentiate
    val = float(loss_fn(net(X_va), y_va))
```

- `parameters()` hands every W and b to the **optimizer**. `weight_decay` is an L2 penalty on the weights: ridge regression, in one argument.
- Gradients *accumulate* across calls to `backward()`: without `zero_grad()` you descend on the sum of every gradient so far.
- Initial weights are a random draw and a **seed** fixes it. `eval()` switches layers such as dropout; `no_grad()` turns the tape off when you only want predictions.
- **Forgetting `zero_grad()` raises no error. It trains the wrong model.**

Source: Session 2 lab notebook, blocks A4 and C.

© Zhigang Feng. Redistribution prohibited without permission

Start at Version 0, Where You Know the Answer

```
torch.manual_seed(0)
X = torch.randn(200, 2)
y = 1.0 + X @ torch.tensor([[2.0], [-0.5]]) + 0.1 * torch.randn(200, 1)
#  ... the five-line loop on nn.Linear(2, 1), 300 steps ...
# OLS    : [ 1.0073  1.9972 -0.5023]
# learned: [ 1.0073  1.9972 -0.5023]
assert np.allclose(ols, learned, atol=1e-3)
```

- A linear “network” trained on squared error must land on the OLS coefficients. It does, to four decimals.
- If it had not, the bug would be in the loop, and you would know before touching a real problem.
- The same habit in Deck 2.5: solve the growth model first where the policy is known in closed form.
- **Version 0 of any project is the case with a known answer.**

Source: Session 2 lab notebook, block A3.

© Zhiguo Feng Redistribution prohibited without permission

Session 2.4

A Curve Whose Answer Is Known

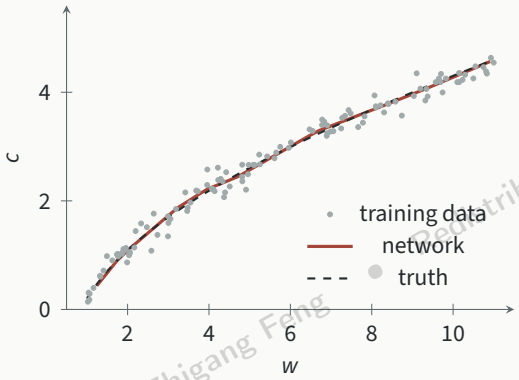
Fit it, then check it on data it has not seen

Redistribution prohibited without permission



© Zhigang Feng

Fit: a Network Finds $c = f(w)$



- A hidden law, $c = \ln w + 0.2\,w + \varepsilon$ on $w \in [1, 11]$, noise variance 0.0225; 300 training and 100 test observations.
- 32 ReLU units, 1,500 Adam steps. The functional form was never specified.
- Mean squared error: 0.0231 in training, 0.0229 on the test data.

Source: Session 2 lab notebook, block B.

© Zhigang Feng — Redistribution prohibited without permission

The Code, Block 1: Make the Data and Split It

```
torch.manual_seed(42)
w = torch.rand(400, 1) * 10 + 1
c = torch.log(w) + 0.2 * w + 0.15 * torch.randn(400, 1)
idx = torch.randperm(400); tr, te = idx[:300], idx[300:]
mu, sd = w[tr].mean(), w[tr].std()
```

- Line 2: 400 draws of *w*, uniform on $[1, 11]$, shape $(400, 1)$: *n* observations of one variable.
- Line 3: the hidden law plus noise with standard deviation 0.15, so the noise variance is $0.15^2 = 0.0225$.
- Line 4: shuffle the row numbers; the first 300 train, the last 100 test. A random split is fine for a cross-section. The yields, a time series, will be split by date.
- Line 5: the scaler's mean and standard deviation come from the training rows only.
- **We wrote the law ourselves, so we know the right answer before fitting anything.**

Source: Session 2 lab notebook, block B.

© Jugans Ref. Redistribution prohibited without permission

The Code, Block 2: the Network and the Training Loop

```
def fit_curve(hidden=32, act=nn.ReLU, epochs=1500, seed=0):
    torch.manual_seed(seed)
    net = SimpleNet(hidden, act)
    opt = torch.optim.Adam(net.parameters(), lr=1e-2)
    for _ in range(epochs):
        loss = ((net((w[tr] - mu) / sd) - c[tr]) ** 2).mean()
        opt.zero_grad(); loss.backward(); opt.step()
    net.eval()
    return net
```

Redistribution prohibited without permission

- Lines 2–4: fix the initial draw; build `SimpleNet` (1 input, 32 ReLU units, 1 output); hand its Ws and bs to Adam.
- Line 6: forward on the standardised inputs, then the mean squared error against c.
- Line 7: clear, backward, update. The five-line loop, 1,500 times on all 300 rows.
- **Nowhere does the code say $\ln w$. The functional form is never written down.**

© Ehigiator Ferry

Source: Session 2 lab notebook, block B.

The Code, Block 3: Score It on Both Samples

```
net = fit_curve()
with torch.no_grad():
    mse = {name: ((net((w[i] - mu) / sd) - c[i]) ** 2).mean().item()
           for name, i in (("train", tr), ("test", te))}
print({k: round(v, 4) for k, v in mse.items()}, "| noise variance = 0.0225")
# {'train': 0.0231, 'test': 0.0229} | noise variance = 0.0225
```

- Line 2: `no_grad()` turns the tape off. We only want predictions now.
- Lines 3–4: the same formula on each sample, picked by its row numbers. The test rows are standardised with the training `mu` and `sd`.
- The noise variance, 0.0225, is the error even the true law would make. No model can beat it on average.
- Training and test errors are close to each other and to that floor.
- **The test number is the one to report: those 100 rows played no part in fitting.**

Source: Session 2 lab notebook, block B; the plotting lines are left out.

© Ehrane Fe... Redistribution prohibited without permission

Session 2.4

Next Month's 10-Year Yield

The data, the question, the split, and the fit



© Zhigang Feng
Redistribution prohibited without permission

The Data: Five FRED Series, Monthly

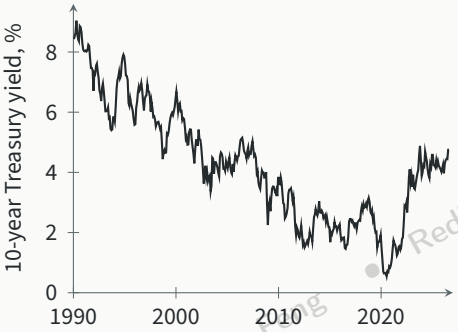
FRED series	What it is	How it enters
DGS10, DGS3MO	10-year and 3-month Treasury yields, daily	last trading day of each month
FEDFUNDS	effective federal funds rate, monthly	value for month $t-1$
UNRATE	unemployment rate, monthly	value for month $t-1$
CPIAUCSL	consumer price index, monthly	12-month inflation, month $t-1$

- Downloaded once, on 20 September 2026: 439 monthly rows, the last for August 2026.
- Yields are market prices, seen at month-end t. Macro series are published with a lag, so the value used at t is the one for month $t-1$.
- **Every input is something a forecaster could have known at the end of month t.**

Source: Federal Reserve Bank of St. Louis, FRED, fred.stlouisfed.org; Session 2 lab notebook, block C.

© Zhigang Feng Redistribution prohibited without permission

The Question

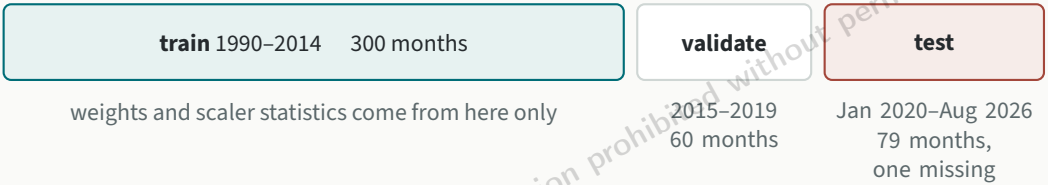


- At the end of month t, predict the 10-year yield at the end of month $t+1$.
- Inputs: the 10-year yield and the 10-year minus 3-month spread at t; funds rate, unemployment and inflation for $t-1$.
- The model: a small network that predicts the *change*; forecast = this month's level + change.
- **The aim is not to beat the bond market, but to fit a real series end to end.**

Source: FRED series DGS10, month-end, downloaded 20 Sep 2026.

© Zhigang Feng Redistribution prohibited without permission

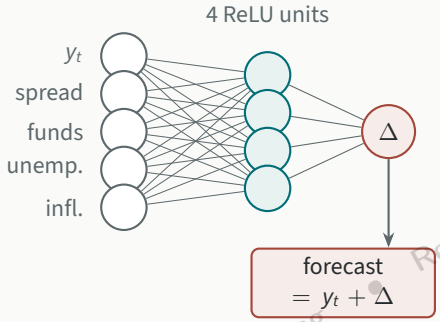
Splitting the Data by Date



© Zhiang Fang · Redistribution prohibited without permission

- Split by the *date of the label*, in order. No shuffle: next month is not a fresh draw.
- **Training** data fit the weights; **validation** data choose the width and when to stop; **test** data are used once, at the end.
- Inputs are standardised with the mean and standard deviation of 1990–2014 only.
- 79, not 80: the December 2025 label is missing, because FRED has no October 2025 CPI or unemployment value.
- **Anything estimated is estimated on the training window.**

The Network Behind the Forecast

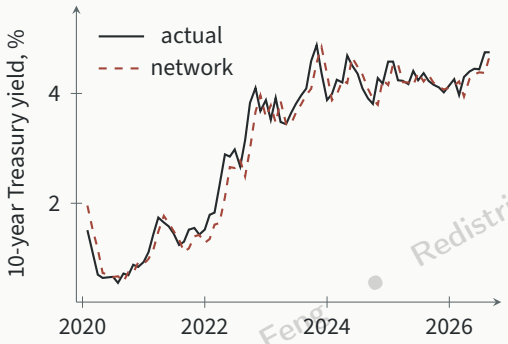


- Five standardised inputs, one hidden layer; the output is the *change*, so zero means “no change”.
- Width picked on validation from 4, 8, 16 (MSE 0.035, 0.047, 0.040): 4 units, 29 parameters.
- Full-batch Adam, step 0.003, weight decay 0.001 (ridge); stop after 40 epochs without a validation gain, keep the best.
- Ten seeds give test RMSEs of 0.27 to 0.36 points; the next slide averages them.
- **Small on purpose: 300 training months cannot pin down many parameters.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block C.

Actual and Network, Month by Month



- The 79 test months, never seen in training. Red: the forecast, averaged over ten seeds.
- It tracks the path through the 2020 low and the 2022 rise.
- Most of the tracking comes from starting at this month's level: the learned changes are small.
- **The training loop from earlier in this deck, unchanged, runs on a real economic series.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block C; FRED series DGS10, month-end, downloaded 20 Sep 2026.

Session 2.5

Solving the Growth Model as Supervised Learning on the Euler Equation

The network is the policy, sampled states are the inputs, and the Euler equation supplies the labels



© Zhigang Feng Redistribution prohibited without permission

Zhigang Feng

Session 2.5

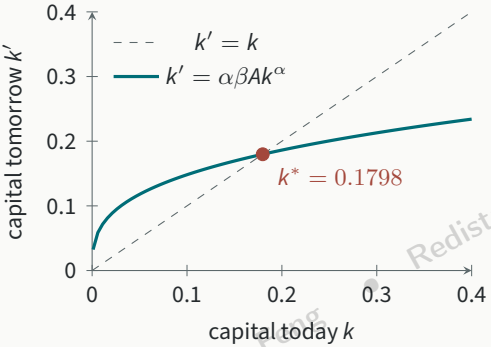
The Model

One unknown function, and the condition it must satisfy



© Zhigang Feng Redistribution prohibited without permission

The Growth Model



© Zhigang Feng
Redistribution prohibited without permission

- Maximise $\sum_t \beta^t u(c_t)$ subject to $c + k' = Ak^\alpha + (1-\delta)k$.
- The answer is a **policy function** $k' = \pi(k)$: capital tomorrow, given capital today.
- Rung 0: log utility, $\delta = 1$, $\beta = 0.96$. The policy is known: $k' = \alpha\beta Ak^\alpha$ (left).
- Rung 1: $u = c^{1-\gamma}/(1-\gamma)$, $\gamma = 2$, $\delta = 0.025$, $\beta = 0.99$. No closed form; $k^* = 28.35$.
- **Solving the model means finding the function π.**

Source: Both rungs: $\alpha = 0.33$, $A = 1$; the lab's names. Brock & Mirman, *JET* 4(3), 1972; the model of Session 1.4.

The Euler Equation, and Its Unit-Free Residual

- The **Euler equation**: at an optimum, giving up one unit of consumption today and eating the proceeds tomorrow must leave utility unchanged,

$$u'(c) = \beta\, u'(c')\, R', \qquad R' = \alpha A k'^{\alpha-1} + 1 - \delta.$$

- Under a candidate policy π, both sides can be computed at any k: $k' = \pi(k)$, c from the budget, then $k'' = \pi(k')$ and c'. Their gap is the **residual**.
- Divide and take logs, so the gap has no units:

$$r_\pi(k) = \log \frac{\beta\, u'(c')R'}{u'(c)}.$$

- $r = 0.01$ reads as a one per cent wedge between the two sides, at any calibration.
- **The true policy makes the residual zero at every k.**

© Zhigang Feng · Redistribution prohibited without permission

Session 2.5

How We Usually Solve It

Time iteration on the Euler equation



© Zhigang Feng
Redistribution prohibited without permission

Time Iteration: the Algorithm

- Grid $k_1 < \cdots < k_N$; resources $f(k) = Ak^\alpha + (1-\delta)k$; a first guess π_0.
- Round n: at each node k_i, solve one equation for today's choice $x = k'$, with tomorrow's behaviour given by last round's policy π_n:

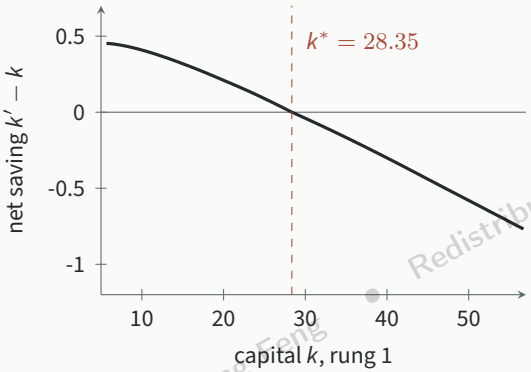
$$u'\big(f(k_i) - x\big) = \beta\, u'\big(f(x) - \pi_n(x)\big)\,\big(\alpha A x^{\alpha-1} + 1 - \delta\big).$$

- Set $\pi_{n+1}(k_i) = x_i$. Between nodes, interpolate.
- Stop when $\max_i |\pi_{n+1}(k_i) - \pi_n(k_i)| / \pi_n(k_i) < 10^{-10}$.
- **Each round solves the Euler equation exactly at the nodes, given the previous round's policy.**

© Zhigang Feng. Redistribution prohibited without permission

Source: W. Coleman, "Equilibrium in a production economy with an income tax", *Econometrica* 59(4), 1991. Lab: 400 nodes, log-spaced; each root by bisection.

Time Iteration's Answer, and Its Cost



- Below k^* the economy saves, above it dissaves: capital moves to the steady state.
- 400 nodes, 292 rounds, 1.5 seconds on one CPU core.
- $\log_{10}$ max Euler error -3.44: one dollar of consumption in about 2,750.
- With d state variables, N nodes per variable become N^d nodes.
- **Fast and accurate in one dimension. This is the answer the network must match.**

Source: Session 2 lab notebook, block D (rung 1).

© Zhigang Feng
Redistribution prohibited without permission

Session 2.5

Reframing It as Supervised Learning

The network is the policy; the Euler equation supplies the labels



© Zhigang Feng
Redistribution prohibited without permission

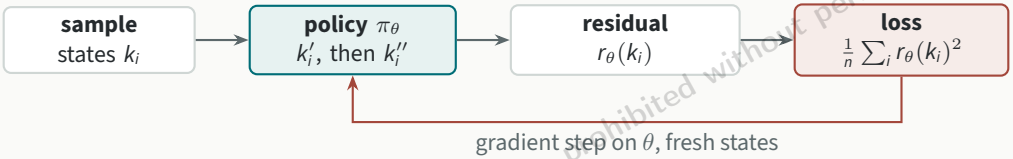
Solving a Model as Supervised Learning: the Dictionary

Supervised learning	Digits (Decks 2.1–2.3)	Growth model (this deck)
inputs x_i	images	sampled capital stocks k_i
model f_θ	classifier	the policy $k' = \pi_\theta(k)$
prediction error	$\hat{p}$ against the label	the Euler residual $r_\theta(k_i)$
label	the true digit	**zero**, at every state

© Zhirang Feng · Redistribution prohibited without permission

- Time iteration hands the Euler equation a grid. Here a network is the policy, and the equation becomes the loss.
- Nobody hands us the right k' at each k. The optimality condition says what a right answer must satisfy.
- **Inputs are free: draw as many states as you like, fresh at every step.**

From Residual to Loss: Zero Is the Label

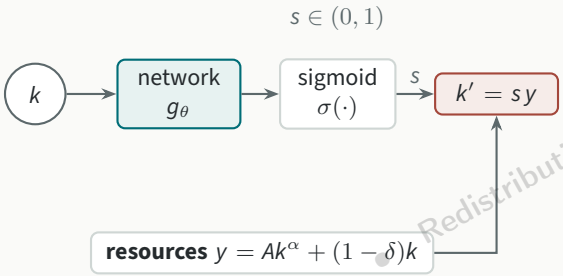


© Zhiyang Feng · Redistribution prohibited without permission

- Squared error against a label of zero: $\ell_i = (r_\theta(k_i) - 0)^2$. The training loop is Deck 2.4's five lines.
- The policy is evaluated twice, at k and at k', so gradients flow through both.
- Network: two tanh hidden layers of 32 units; 256 fresh states per step; Adam, 500 steps on rung 0, 3,000 on rung 1.
- **No benchmark policy ever enters the loss. Benchmarks are kept for testing.**

Source: Session 2 lab notebook, block D. Azinovic, Gaegauf & Scheidegger, "Deep equilibrium nets", *IER* 63(4), 2022.

Feasibility by Construction: the Sigmoid Share



© Zhigang Feng Redistribution prohibited without permission

- **Feasibility**: $c > 0$ and $k' > 0$. One infeasible state makes $\log c$ undefined and ends training.
- The network outputs a *saving share* $s = \sigma(g_\theta) \in (0, 1)$, so $k' = s\,y$ and $c = (1 - s)\,y$ are positive at every θ.
- The input is $\log k$, rescaled to $[-1, 1]$.
- **A constraint written into the architecture cannot be violated, and costs no penalty term.**

The Code: Policy and Residual

```
def saving_share(self, k):
    x = (torch.log(k) - self.log_center) / self.log_scale
    return torch.sigmoid(self.net(x))

def forward(self, k):
    return self.resources(k) * self.saving_share(k)

def log_residual(model, k):
    c, rhs = euler_rhs(model, k)
    return torch.log(rhs) + model.rung.gamma * torch.log(c)

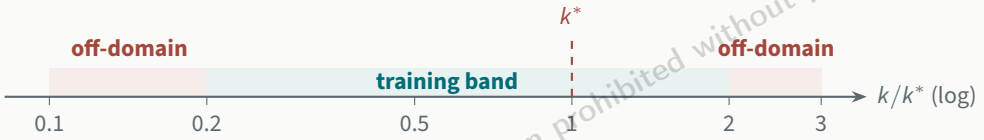
loss = log_residual(model, k).square().mean()
```

- euler_rhs returns c and $\beta u'(c')R'$; with $u'(c) = c^{-\gamma}$, line 10 is $\log \text{RHS} - \log u'(c)$.
- **Feasibility is line 6; the economics is line 10.**

Source: Session 2 lab notebook, block D, shown without the productivity shock.

© Zhenyang Peng Redistribution prohibited without permission

Where to Sample



Redistribution prohibited without permission
© Zixiang Feng

- **Training** states: uniform in $\log k$ on $[0.2,\ 2] \times k^*$, fresh at every step.
- **Held-out** states: new draws from the same band, from a generator training never used. They test coverage.
- **Off-domain** states: $[0.1, 0.2) \cup (2, 3] \times k^*$, never asked of the network. They test extrapolation.
- **The solution is claimed only where it was trained and tested.**

Session 2.5

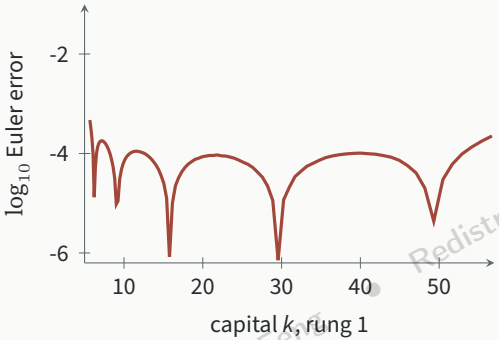
How We Test It

Residuals where it was not trained, the known answer, and the steady state



© Zhigang Feng Redistribution prohibited without permission

Held-Out and Off-Domain Residuals, on Judd's Scale



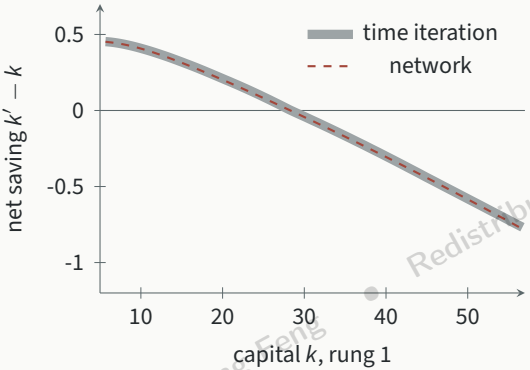
$\log_{10}$ max error	rung 0	rung 1
held-out	-1.95	-3.34
off-domain	-1.29	-1.59

- **Judd's scale**: $\log_{10}$ of the consumption error implied by the residual. -3 is one dollar in a thousand.
- Off-domain, accuracy falls by more than half a decade on rung 0 and almost two on rung 1.
- **Report the maximum, and report it where the network was not trained.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block D. Judd, *JET* 58(2), 1992.

The Network Against the Known Answer



© Zhigang Feng
Redistribution prohibited without permission

- Benchmarks: the closed form (rung 0), time iteration (rung 1). Neither entered training.
- Largest policy error on held-out states: 0.64% on rung 0, 0.04% on rung 1.
- Off-domain: 3.4% and 1.3%.
- On rung 1 the saving share starts at its steady-state value; the lab shows a default start.
- **The two curves lie on top of each other.**

Source: Session 2 lab notebook, block D.

The Steady-State Check, and the Gates

Gate	rung 0	rung 1
feasible at every tested state	yes	yes
max policy error, held-out (gate: below 1%)	0.64%	0.04%
max policy error, off-domain (reported)	3.4%	1.3%
simulated against analytical k^* (gate: 1%)	0.47%	0.18%

© Zhitang Fang · Reproduction prohibited without permission

- Steady-state check: from $0.5\,k^*$, apply the learned policy 3,000 times. A *global* test: small errors that compound along the path show up here.
- A **gate** is a pass mark fixed before the run.
- **Both rungs pass every gate.**

Source: Session 2 lab notebook, block D; policy errors against the closed form (rung 0) and time iteration (rung 1).

Time Iteration and the Network, Side by Side

Rung 1	time iteration	network
numbers stored	400 grid values	1,153 parameters
seconds, one CPU core	1.5 (292 iterations)	4.6 (3,000 steps)
$\log_{10}$ max Euler error, held-out band	-3.44	-3.34

Redistribution prohibited without permission

- In one dimension the classical method ties on accuracy and wins on the rest.
- A grid costs N^d; the network's cost grows with the sample. Its case is dimension.
- **Learn the method where it loses, so that you can trust it where nothing else runs.**
- Next, Deck 2.6: the policy was a plain dense network. What if the architecture itself carried the economics?

Source: Session 2 lab notebook, block D (rung 1: CRRA $\gamma = 2, \delta = 0.025, \alpha = 0.33, \beta = 0.99$).

Session 2.6

Architecture as a Constraint on W, and What More Parameters Buy

Restrictions that encode what we know about the input, and the opposite move: more parameters than observations



© Zhigang Feng Redistribution prohibited without permission

Zhigang Feng

The Same Layer, with Restrictions

- Every network so far has been **dense**: every entry of every W free, nothing assumed about the input.
- A dense layer is indifferent to its input's structure. Shuffle the pixels of every digit the same way and it learns exactly as well — which means it never used the fact that pixels have neighbours.
- **An architecture is a constraint on W**: entries fixed at zero, entries forced equal, entries forced non-negative. Each one is a claim about the target.
- Three of those claims are worth an economist's time: locality (convolutions), shape (monotone and convex policies), and order — or its absence (sequences, networks of agents).
- **Then the opposite question: what does an *un*restricted network buy from having more parameters than observations?**

© Zhigang Feng · Redistribution prohibited without permission

Session 2.6

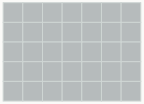
Architecture as a Constraint on W

The same layer, with restrictions that encode what we know about the input

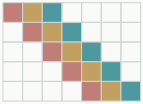


© Zhigang Feng Redistribution prohibited without permission

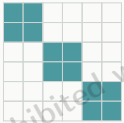
One Idea, Three Times



dense
every entry free



convolution
sparse and repeated



recurrent
one block at every date



graph
follows the links

Redistribution prohibited without permission

- A layer is $\sigma(Wx + b)$. White cells are fixed at zero; in the dense matrix every grey cell is free; in the other three, cells of one colour are *one* shared parameter.
- A dense W assumes nothing about the input, so everything must be learned from data.
- **A constraint on W is a prior about the target. True, it saves parameters and data. False, it is bias.**

Source: Miao & Feng, *AI for Economists*, §9.6.

Session 2.6

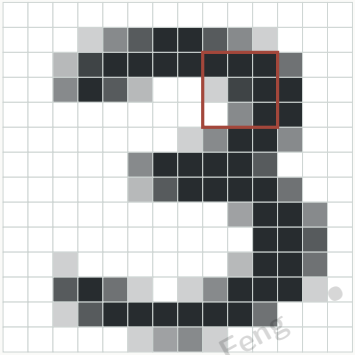
Locality: the Convolution

Two constraints on W — sparse, and repeated



© Zhigang Feng Redistribution prohibited without permission

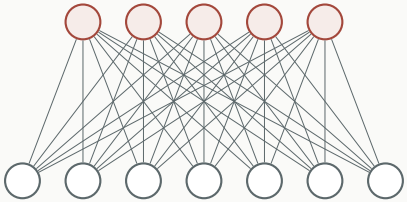
Back to the Digit: What a Dense Layer Ignores



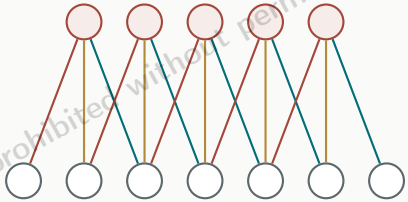
- Deck 2.1 flattened the image into a vector. A dense first layer then has one weight per pixel per hidden unit: $196 \times 128 = 25{,}088$ weights for the 14×14 digit here, $784 \times 128 = 100{,}352$ for MNIST's 28×28.
- Shuffle the pixels, the same way in every image, and a dense network learns exactly as well. It never knew which pixels were neighbours.
- Yet a stroke is a local object: the red 3×3 patch means the same thing wherever it sits.
- **Two facts about images are unused: nearby pixels belong together, and a pattern can appear anywhere.**

© Zhigang Feng
Redistribution prohibited without permission

Dense vs Convolution: 35 Weights vs 3



dense: 35 lines, 35 free weights



convolution: 15 lines, 3 free weights

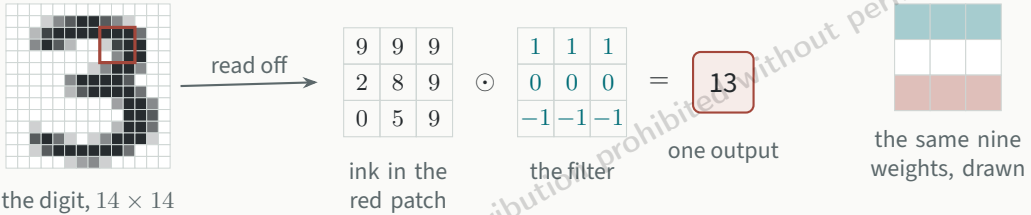
Redistribution prohibited without permission

© Zhiguang Feng

- Seven inputs, five outputs. On the right each output sees three adjacent inputs (**local connection**), and lines of one colour are one parameter (**weight sharing**).
- The three shared weights are the **filter**, or **kernel**. Sliding it along the input is a **convolution**.
- **Convolution is a dense layer after two constraints on W: sparse, and repeated.**

Source: Miao & Feng, *AI for Economists*, §9.6, Fig. 9.9.

The Receptive Field

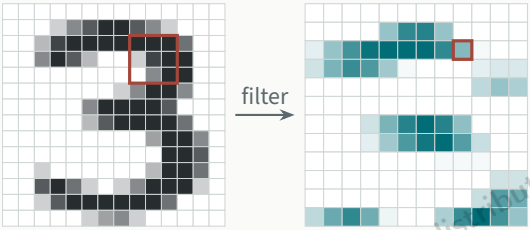


© Zhigang Feng Redistribution prohibited without permission

- The **receptive field** of an output is the part of the input it can see: here the 3×3 patch outlined in red, and nothing else.
- The output is an inner product: $(9+9+9)-(0+5+9)=13$. The filter asks whether there is ink above and paper below, so it fires on the lower edge of a horizontal stroke.
- **Drawn as a picture it is the template neuron of Deck 2.1, shrunk to nine weights — and those nine are reused at every position on the next slide.**

Source: Patch: rows 3 to 5, columns 9 to 11 of the 14×14 digit of Deck 2.1.

Feature Maps: One Question Asked Everywhere

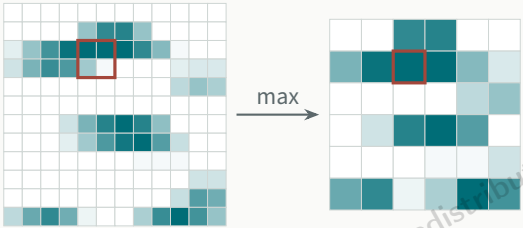


- Slide the filter over every position, apply a ReLU, and record the answers: a **feature map**.
- It lights up along the lower edges of horizontal strokes, wherever they are.
- A layer has many filters, often 32 or 64, each with its own map. They are learned, not designed.
- **One filter is one question, asked at every location with the same nine weights.**

Source: Computed from the digit of Deck 2.1 with the filter of the previous frame; the red cell is the value 13.

© Zhigang Feng Redistribution prohibited without permission

Pooling

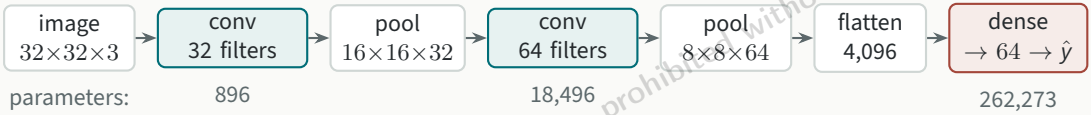


- **Pooling** replaces each small block by one summary number. **Max pooling** keeps the largest.
- The red 2×2 block holds $27,\ 27,\ 10,\ 1$; it becomes 27. Resolution halves: $12 \times 12 \to 6 \times 6$.
- The question becomes "is the edge somewhere in this block?" Pooling has no parameters.
- **Convolution extracts features; pooling reduces resolution, trading location for robustness.**

© Zhigang Feng
Redistribution prohibited without permission

The Full Pipeline: Equivariance, then Approximate Invariance

A colour image now, not the digit: three channels, 32×32 pixels.



- **Equivariance**: shift the input one pixel and the feature map shifts one pixel (weight sharing).
- **Invariance**: the output does not change. Pooling gives it approximately, for small shifts.
- Of 281,665 parameters the convolutions hold 6.9%; the dense head holds 93.1%.
- **A convolutional network is a learned feature extractor followed by the network you already know.**

Source: Miao & Feng, *AI for Economists*, §9.6, Fig. 9.10.

© Chaofeng Feng Redistribution prohibited without permission

What the Constraint Buys, and What It Assumes

- A 256×256 colour image is $196{,}608$ numbers: a dense layer into 256 units needs about 5×10^7 weights, while sixty-four 3×3 filters over three colour channels need $64 \times (3 \cdot 3 \cdot 3 + 1) = 1{,}792$ — four orders of magnitude fewer.
- Fewer parameters need fewer observations. In economics the sample is what it is, so this is decisive.
- The assumption bought: nearby inputs are related, distant ones are not, and a pattern means the same thing everywhere.
- Apply a convolution to data with no spatial order, such as the columns of a survey, and the constraint is simply wrong.
- Random, untrained filters with a ridge regression on top match deep training on satellite tasks (Rolf et al., 2021): the locality and the sharing do the work.
- **Parameters saved are paid for with an assumption. Check the assumption, not the parameter count.**

© Thiago Ferreira · Redistribution prohibited without permission

Source: E. Rolf et al., *Nature Communications* 12, 2021; Miao & Feng, *AI for Economists*, §9.6.

Session 2.6

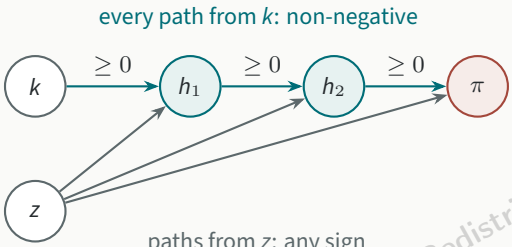
Shape You Can Guarantee

Monotone and convex by construction, and what the guarantee costs



© Zhigang Feng Redistribution prohibited without permission

Monotone Policies: Which Weights Carry the Sign

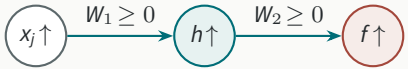


- Theory often gives the shape: saving rises in wealth, demand falls in own price.
- A **monotone network**: every weight on every path from x_j non-negative and activations non-decreasing give $\partial f / \partial x_j \geq 0$ everywhere.
- *Partial* monotonicity is the useful case: constrain the paths from k, leave those from z.
- **Impose $W \geq 0$ by training V in $W = \mathrm{softplus}(V)$; a clip stalls the gradient at the boundary.**

Source: J. Sill, "Monotonic networks", *NeurIPS* 10, 1998.

© Zhigang Feng
Redistribution prohibited without permission

Monotone Networks: Why the Sign Survives, and Where Theory Supplies It



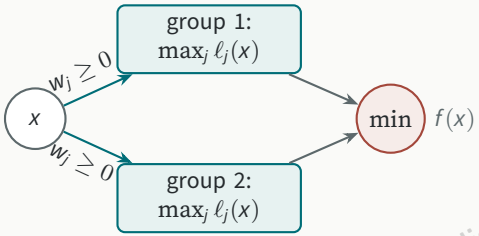
each layer: a larger input gives a larger pre-activation, and a non-decreasing σ passes it on

- **Both conditions are needed.** One negative weight on a path from x_j breaks it; so does a non-monotone activation (GELU and Swish dip below zero).
- **Decreasing instead**: feed $-x_j$, e.g. minus own price for a demand curve.
- **Where theory gives the sign**: saving $a'(a, z)$ rising in assets; consumption rising in cash on hand; demand falling in own price; a value function rising in the resource state.
- **What it prevents**: the spurious wiggles of an unconstrained fit, which a residual plot can hide.
- **Clip or reparameterise?** A weight clipped at zero gets no gradient and stays dead; $W = \text{softplus}(V)$ does not.

Source: J. Sill, *NeurIPS* 10, 1998; Miao & Feng, *AI for Economists*, ch. 9.

© Zhigang Feng. Unauthorized distribution prohibited without permission

What the Sign Constraint Costs, and What It Does Not



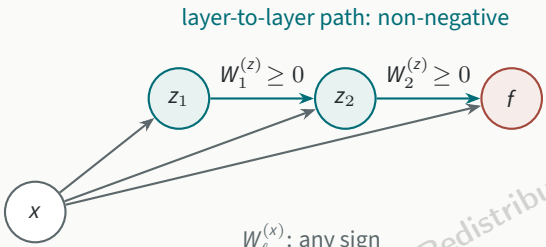
$\ell_j(x) = w_j x + b_j$:
Sill's min–max network

- Sign constraints shrink the parameter set, so a constrained network often trains more slowly and to a *higher* training loss.
- That is a harder search, not a smaller class: maxima within groups of linear units, a minimum across groups, and the family is universal among monotone functions.
- Penalising violations instead stays flexible, and holds only where it was sampled.
- **The unconstrained network reaches its lower loss partly with functions the theory forbids.**

Source: J. Sill, "Monotonic networks", *NeurIPS* 10, 1998, §2 (the min–max architecture).

© Zhigang Feng
Redistribution prohibited without permission

Input-Convex Networks: Which Weights, Which Arguments, Which Direction



- An **input-convex network**: $z_{\ell+1} = \sigma\left(W_\ell^{(z)} z_\ell + W_\ell^{(x)} x + b_\ell\right)$.
- Convex in x if $W_\ell^{(z)} \geq 0$ and σ is convex and non-decreasing (ReLU, softplus). The direct weights $W_\ell^{(x)}$ are free.
- Concave: take $-f$. Convex in some arguments only: constrain only their paths.
- **Examples: a cost function convex in input prices; a production function concave in capital.**

© Zhigang Feng
Redistribution prohibited without permission

Source: B. Amos, L. Xu, J. Z. Kolter, "Input convex neural networks", *ICML*, 2017.

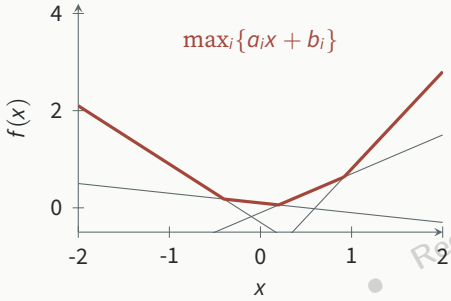
Why the Network Is Convex: Three Facts and an Induction

- **Base.** $z_0 = x$ is linear in x, hence convex.
- **Step.** Suppose every unit of z_ℓ is convex in x. Then
  - $W_\ell^{(z)} z_\ell$ is a *non-negative* combination of convex functions: convex;
  - adding the affine term $W_\ell^{(x)} x + b_\ell$, of any sign, keeps it convex;
  - a convex, non-decreasing σ of a convex function is convex.
- So every layer, and the output f, is convex in x, at every value of the parameters.
- **Both conditions carry weight.** A negative $W^{(z)}$ flips a convex unit to concave; a non-monotone σ such as $\tanh$ breaks the last step.
- **The free direct weights are why it can still fit**: each layer sees x again and can tilt and place the function.

© Zhening Fang · Redistribution prohibited without permission

Source: B. Amos, L. Xu, J. Z. Kolter, "Input convex neural networks", *ICML*, 2017, Prop. 1.

Does Convexity Cost Flexibility? A Shape, Not a Functional Form



- **Theorem**: an input-convex network approximates any convex function on a compact domain.
- Why: a maximum of affine functions is convex (left), every convex shape is a limit of such maxima, and a ReLU input-convex network is a nested max-affine map.
- No CES, Cobb–Douglas or translog: it imposes the *shape*; the data choose within it.
- **The economist's usual position: theory pins the shape, not the curvature.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Y. Chen, Y. Shi, B. Zhang, "Optimal control via neural networks: a convex approach", *ICLR*, 2019. Figure: schematic.

What the Constraint Refuses, and Partial Convexity

- Point the same input-convex network at $\sin 3x$ on $[-1, 1]$: median RMSE 0.345, and the fit stays convex to within 9×10^{-11}. It cannot bend, which is the guarantee seen from the other side.
- **A structured network that cannot fit is saying the imposed theory is wrong**: an informative failure.
- A value function $V(k, z)$ is concave in the endogenous state k and of no particular shape in the shock z. Impose convexity on z too and you add bias that no residual reveals.
- **Partially input-convex** network: $z_{\ell+1} = \sigma\big(W_\ell^{(z)} z_\ell + W_\ell^{(u)}(u \circ v) + W_\ell^{(v)} v + b_\ell\big)$, $W_\ell^{(z)} \geq 0$; capital is the constrained input u, the shock the free input v.
- Concave V makes $\max_{k'}\{u(c) + \beta V(k')\}$ one basin, so gradient steps find its maximum (Session 3).
- **Impose the restriction exactly where theory gives one, and nowhere else.**

Source: Amos, Xu & Kolter, *ICML*, 2017, §3.2. $\sin 3x$: instructor's quantitative-macro course computations (2026), not recomputed here.

© Zhigang Feng · Redistribution prohibited without permission

Measured: What an Unconstrained Network Does to Convexity

	median RMSE	worst negative curvature
plain ReLU network	0.00172	5.93 (range 0 to 10.4 over seeds)
input-convex network	0.01345	below 7.1×10^{-11}

- Target: $0.6x^2 + 0.25x + 0.1$, exactly convex, curvature 1.2 everywhere. 1,153 parameters, five seeds.
- The plain network fits the *level* eight times better and is locally concave by about 5.9. Its loss shows none of that.
- At one seed in five it came out convex anyway: you cannot tell without checking.
- **The constrained network buys a guarantee and pays for it in fit: 7.8 times the RMSE.**

Source: Recorded in the instructor's quantitative-macro course computations (2026); not recomputed in this session's lab.

© Thiago Ferreira. Redistribution prohibited without permission

What Structure Buys, and What It Refuses

- **Buys reliability.** The shape holds at every θ, every seed, and outside the training band, where data cannot enforce it.
- **Buys well-behaved downstream problems.** Convex costs give monotone factor demands, so a market-clearing price is unique and a calibration loop converges.
- **Buys data efficiency.** Fewer functions to choose between means fewer states to pin one down.
- **Refuses flexibility.** If the true function violates the shape, the constraint is bias. It must come from theory, not convenience.
- Session 3 uses the same device for value functions.
- **Deck 2.5 imposed exactly one of these, feasibility, because it is the one theory guarantees on every rung.**

© Zigang Feng · Redistribution prohibited without permission

Session 2.6

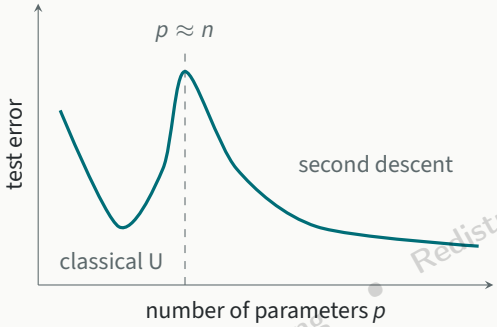
What More Parameters Buy

The opposite move: no constraint at all, and more parameters than observations



© Zhigang Feng Redistribution prohibited without permission

The Modern Twist: Double Descent

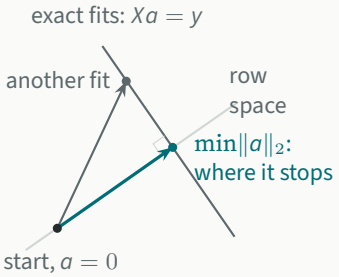


- Test error follows the textbook U, then peaks at the **interpolation threshold** $p \approx n$, where the model first fits the sample exactly.
- With $p \gg n$ it falls again: **double descent** (a schematic).
- In forecasting, $p > n$ can lower out-of-sample risk.
- **More parameters than observations is not a defect. It is a claim to test out of sample.**

© Zhigang Feng Redistribution prohibited without permission

Source: Measured on this deck's own example four frames on. M. Belkin, D. Hsu, S. Ma, S. Mandal, *PNAS* 116(32), 2019; Y. Liao, X. Ma, A. Neuhierl, Z. Shi, "Economic forecasts using many noises", arXiv 2312.05593, 2023.

Why Gradient Descent Picks That Fit

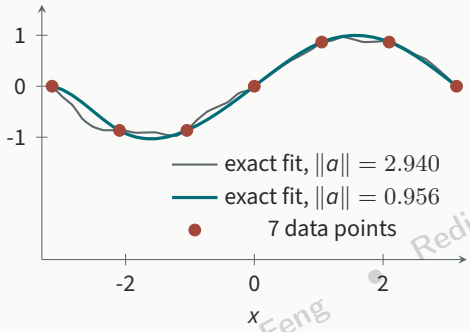


- Linear in the parameters: every gradient $\nabla_a L = X^\top(Xa - y)$, and so every step, lies in the **row space** of X.
- From $a = 0$ the iterate never leaves it, and one exact fit alone lies there: the smallest $\|a\|_2$. **The optimiser cannot move in directions the data never mention.**
- $\|\cdot\|_2$ is the selector, Euclidean because the steps are — not a smoothness claim. Multiplicative updates select ℓ_1 instead: the algorithm is part of the estimator.
- **With $p > n$ the algorithm picks the fit. Whether it generalises is empirical, not a theorem.**

© Zhigang Feng
Redistribution prohibited without permission

Source: The min-norm fact above is elementary: the derivation is its proof. S. Gunasekar, J. Lee, D. Soudry, N. Srebro, “Characterizing implicit bias in terms of optimization geometry”, ICML 2018.

Implicit Regularization, Stated Precisely

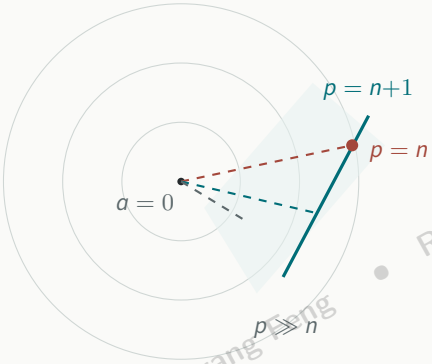


- Seven points of $\sin x$, 41 fixed features: the coefficients $a \in \mathbb{R}^{41}$ are not pinned down, and infinitely many fits are exact.
- Gradient descent from zero converges to the fit of **smallest Euclidean norm** (teal). No penalty was written.
- **The optimiser picks among exact fits. For deep networks: intuition, not a theorem.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block B3. Soudry et al., *JMLR* 19, 2018 (the classification analogue: the iterate's *direction* converges to the maximum margin); Hastie et al., *Ann. Statist.* 50(2), 2022 (risk at $p > n$).

Why the Norm Moves: It Is a Minimum over a Set



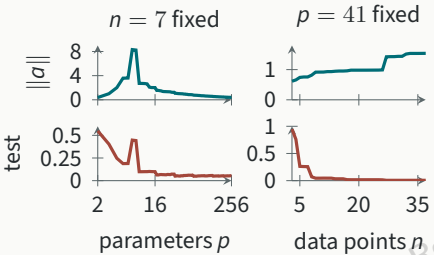
Dashed: the smallest member of each set.

- $\|a\|$ is the *smallest* coefficient vector that fits. It is a minimum over a set, so whatever changes the set changes it.
- **More parameters** give more ways to fit: the set grows, so its minimum falls. **More data** give more to satisfy: the set shrinks, so its minimum rises.
- At $p = n$ the set is a *single point* — nothing to choose — and that is the peak. Below $p = n$ nothing fits exactly at all.

Source: Both directions follow from one argument; both are checked in the Session 2 lab notebook, block B3.

© Zhirang Yang Redistribution prohibited without permission

Two Levers, Measured

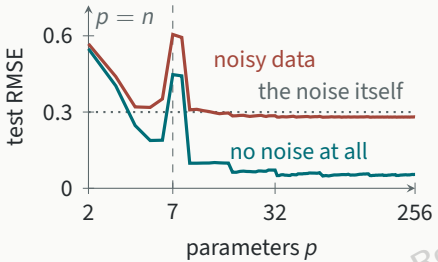


- **More** p (left): below $p = n$ nothing fits exactly; at $p = n$ the single fit costs 8.3; above it the norm falls to 0.4.
- **More** n (right): $0.62 \to 1.54$, and every fit is still exact.
- Test error follows neither: past $p = n$ it drops — the optimiser may now *choose* — then stalls near $p = 65$. More data never stops helping, $0.95 \to 0.006$.
- **The norm falls with the lever that need not help, and rises with the one that always does.**

Source: Session 2 lab notebook, block B3. Nested hinges and nested data points, so each sweep adds to what came before.

© Zhigang Feng
Redistribution prohibited without permission

Is That Just Overfitting?



- No: the training error is **zero for every $p \geq n$ on both curves**, so a loss below the noise floor cannot tell them apart.
- **Overfitting** drags the model onto noise. Here every candidate ties at zero; only the tie-break differs.
- With $y = \sin x$ exactly there is nothing to overfit, yet $p = n$ gives 0.45 against 0.19 at $p = 6$. With noise the two stack and the peak rises to 0.61.
- **Overfitting is cured with less capacity; this, with a better tie-break or more data.**

Source: Session 2 lab notebook, block B3; $n = 7$, noise s.d. 0.30, mean over 200 draws.

© Zhigang Feng
Redistribution prohibited without permission

Session 2.6

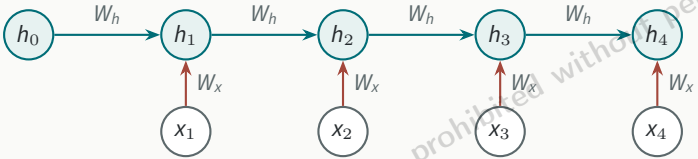
Order, and Its Absence

Sequences, and sets of agents where names do not matter



© Zhigang Feng Redistribution prohibited without permission

Recurrent Networks: the Hidden State Is a State Variable

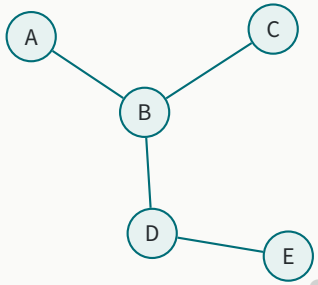


© Zhiguo Feng — Redistribution prohibited without permission

- A **recurrent network** reads a sequence one date at a time: $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$, the same W_h, W_x at every date.
- The **hidden state** h_t summarises the history: a state-space model with an estimated transition.
- Going back T dates multiplies by W_h^T, which vanishes or explodes. An LSTM's gates make persistence depend on the state; an AR(1) has a constant ρ.
- **The constraint: one law of motion at every date.**

Source: S. Hochreiter and J. Schmidhuber, "Long short-term memory", *Neural Computation* 9(8), 1997; Miao & Feng, §9.6.

Graph Networks: Permutation Invariance Is an Economic Restriction



five banks; an edge is lending

- A **graph network** updates each node from itself and its neighbours' average, with one pair of matrices for all nodes: $h'_v = \sigma\big(W_{\mathrm{s}} h_v + W_{\mathrm{n}}\, \overline{h}_{N(v)} + b\big)$.
- Neighbours have no order, so the aggregate is a sum or a mean: **permutation invariance**. Renumbering nodes cannot change the answer.
- **Heterogeneous-agent models impose the same: prices depend on the distribution, not on who is who (Session 4).**

© Zhigang Feng
Redistribution prohibited without permission

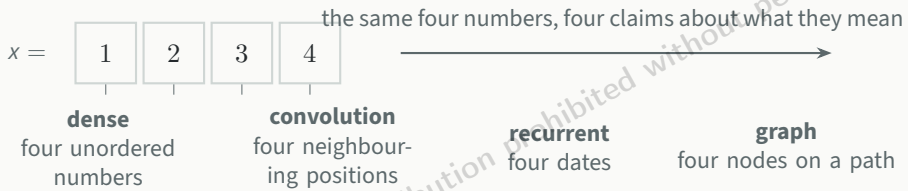
Source: T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks", ICLR 2017; Miao & Feng, §9.6.

Takeaways

- **An architecture is a prior about the input.** True, it saves parameters and data. False, it is bias.
- **A convolution is a dense layer after two constraints**: local, and the same filter everywhere.
- **Sign constraints on W buy a guarantee and pay for it in fit**: 7.8 times the RMSE on the convexity test, and a harder search — not a smaller class.
- **Shape is the constraint an economist can usually justify**, because theory delivers monotonicity and convexity before any data do.
- **More parameters than observations is not overfitting**: every candidate ties at zero training error, and only the tie-break differs.
- **Both halves of this deck are the same question: what decides the answer when the data do not?**

© Zhaoning Feng · Redistribution prohibited without permission

Appendix: One Input, Read Four Ways



© Zhiang Feng ● Redistribution prohibited without permission

- Every layer below is the same $\sigma(Wx + b)$. They differ only in which entries of W are free, which are fixed at zero, and which are forced to be equal.
- Weights are chosen here, not trained, so every number on the next two pages can be checked by hand.
- **The input never changes. The architecture is the claim about what its four numbers are.**

Appendix: Four Layers, Four Answers

	Rule	Output	Free
dense	$w = (\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}, -\frac{1}{2})$	-1	5
convolution	filter $(1, -1)$, slid	$(-1, -1, -1)$	3
recurrent	$h_t = \frac{1}{2} h_{t-1} + x_t$	$(1,\ 2.5,\ 4.25,\ 6.125)$	3
graph	$h'_v = x_v + \frac{1}{2} \overline{x}_{N(v)}$	$(2,\ 3,\ 4.5,\ 5.5)$	3

1	−1	0	0
0	1	−1	0
0	0	1	−1

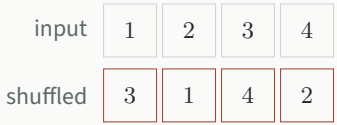
the convolution's W:
3×4, three numbers

- Dense returns one number, the convolution three, the recurrent and graph layers four: the constraint changes the *shape* of the answer, not only the count of parameters.
- The convolution's W is written out on the right. Nothing was added to the dense layer — entries were set to zero and the survivors tied together.
- **Three free numbers do the work of five, and only because the tying is true of the input.**

Source: Every value asserted against the arithmetic in the Session 2 lab notebook, appendix (one input, four architectures).

© Shigaev Petr. Redistribution prohibited without permission

Appendix: What Each One Refuses to Notice



convolution: $(-1, -1, -1)$

convolution: $(2, -3, 2)$

the dense layer, with its weights relabelled the same way, is unchanged

© Zhirang Feng Redistribution prohibited without permission

- **Shuffle the input.** The convolution's answer is destroyed: it assumed positions 1 and 2 were neighbours, and now they are not. The dense layer never made that claim, so it learns the shuffled problem just as well — the digit slide's point, as arithmetic.
- **Relabel the graph's nodes** *A B C D* as *D C B A*: the output becomes $(5.5,\ 4.5,\ 3,\ 2)$, the same four numbers in the new order, and a sum readout stays 15 either way. That is permutation invariance, as a number.
- The recurrent layer weights the past geometrically, so any reordering of the dates changes the answer: order is the whole of its claim.
- **A constraint is not a saving. It is a statement about the input that can be wrong.**